# Project Report

On

# “Luxelle - Online Fashion & Lifestyle Selling”

Submitted in Partial Fulfillment for Degree of

## “Bachelor of Computer Application”


## Submitted By

**Mr. Suraj V. Manani**

**Mr. Raj M. Mandaliya**

## Guided By:

**Mrs. Dhvani Kanoje**


## 2025 – 2026


VIDHYADHAN CHARITABLE TRUST

**SMT Z.S Patel College of Computer Application**

Affiliated To

**Veer Narmad South Gujarat University, Surat**

# Acknowledgement

I would like to express my sincere gratitude to everyone who supported me throughout the completion of this project.

First and foremost, I would like to thank my guide/supervisor **Dhvani Kanoje Ma'am** for their valuable guidance, continuous support, and constructive feedback, which greatly helped me in completing this project successfully.

I would also like to extend my thanks to **SMT Z.S Patel College of Computer Application** for providing the necessary resources, facilities, and environment to carry out this work.

My heartfelt appreciation goes to my faculty members, friends, and classmates for their encouragement, suggestions, and assistance whenever needed.

Finally, I would like to thank my family for their constant motivation, patience, and moral support throughout this journey.

**Place:** Surat

**Signature:**

**Date:**

# Index


| Sr. No. | Title | Page No. |
|---|---|---|
| **1.** | **Introduction:** | **4** |
| | 1.1 Project Profile | 5 |
| | 1.2 Overview of Project | 6 - 7 |
| | 1.3 Problem Statement | 8 – 9 |
| | | |
| **2.** | **Proposed System:** | **10** |
| | 2.1 Existing System & Limitations | 11 - 12 |
| | 2.2 Objectives | 13 – 14 |
| | 2.3 Hardware and Software Platforms | 15 |
| | 2.4 Scope | 16 – 17 |
| | 2.5 Feasibility Study | 18 - 19 |
| | | |
| **3.** | **System Design:** | **20** |
| | 3.1 System Architecture | 21 – 22 |
| | 3.2 Data Flow diagram | 23 - 24 |
| | 3.3 UML Diagram | 25 – 28 |
| | 3.4 Data Dictionary | 29 - 32 |
| | 3.5 Interface Design (Screenshots) | 33 – 50 |
| | | |
| **4.** | **System Testing:** | **51** |
| | 4.1 Fronted (Client-Side) Validation | 52 |
| | 4.2 Backend (Server-Side) Validation | 53 |
| | 4.3 Authentication & Authorization Validation | 54 |
| | 4.4 Manual Testing | 55 |
| | | |
| **5.** | **Results** | **56 - 58** |
| | | |
| **6.** | **Future Enhancement** | **59 – 61** |
| | | |
| **7.** | **Bibliography** | **62 - 65** |

# Chapter 1: Introduction

# 1.1 Project Profile

| | |
|---|---|
| **Project Title** | Luxelle |
| **Domain** | Fashion & Lifestyle E-Commerce |
| **Technology Stack** | MERN Stack (MongoDB, Express.js, Angular, Node.js) |
| **Fronted** | Angular (v21.1.0), Tailwind CSS (v3.4.19) |
| **Backend** | Node.js, Express.js |
| **Database** | MongoDB |
| **Tools** | VS Code, Git |

## 1.2 Overview of Project

Luxelle serves as a comprehensive, state-of-the-art e-commerce platform specifically tailored for the modern fashion retail sector. In an era where digital presence is synonymous with brand identity, Luxelle bridges the gap between high-end aesthetic appeal and robust, scalable technical functionality. The platform is designed to offer a seamless shopping experience that mimics the exclusivity and personal touch of a luxury boutique, translated into a digital medium. By leveraging modern web technologies, it ensures that users enjoy fast load times, smooth transitions, and an intuitive interface that encourages exploration and purchase.

The core philosophy behind Luxelle is "Luxury First." This design principle influences every aspect of the user interface, from the sophisticated typography and curated color palettes to the high-resolution imagery and minimalist layout. Unlike generic e-commerce templates that often feel cluttered or utilitarian, Luxelle prioritizes visual harmony and effective use of whitespace. This approach not only elevates the perceived value of the products displayed but also reduces cognitive load for the user, allowing them to focus entirely on the merchandise. The result is a digital storefront that feels premium, trustworthy, and engaging. From a technical perspective, Luxelle is built upon the powerful MEAN stack (MongoDB, Express.js, Angular, and Node.js), demonstrating a commitment to using cutting-edge, full-stack JavaScript technologies. This architecture provides a unified development environment that enhances efficiency and performance. The use of Angular on the frontend ensures a dynamic Information Single Page Application (SPA) experience, where content updates instantly without full page reloads. Meanwhile, the Node.js and Express.js backend offers a non-blocking, event-driven environment capable of handling concurrent user requests with high throughput and low latency.

Beyond the customer-facing interface, Luxelle includes a robust administrative dashboard that serves as the command center for business operations. This secure backend portal empowers administrators to manage the entire lifecycle of their e-commerce business with ease. Features include comprehensive product management tools for adding, editing, and categorizing inventory, as well as real-time order tracking that monitors purchases from placement to delivery. This centralization of operational control streamlines daily tasks, reducing the administrative burden and allowing business owners to focus on growth and strategy.

Security and data integrity are paramount in the Luxelle ecosystem. The platform implements industry-standard security measures, including JSON Web Token (JWT) authentication, to ensure that user sessions are secure and private. Sensitive data, such as user passwords, are hashed and encrypted before storage, complying with best practices for data protection.

Finally, Luxelle is designed with scalability and future growth in mind. The modular architecture allows for easy integration of new features and third-party services as the business evolves. Whether it is adding new payment gateways, integrating advanced analytics tools, or expanding the product catalog to include thousands of items, the underlying system is engineered to handle increased load and complexity without compromising performance.

## 1.3 Problem Statement

The modern fashion industry faces a significant challenge in the digital realm: the "Generic Template" problem. Most existing e-commerce solutions prioritize utility over aesthetics, resulting in online stores that feel transactional and cold. This lack of visual storytelling often dilutes the brand value of high-end fashion labels, making it difficult for them to differentiate themselves in a saturated market. Luxury products require a digital environment that reflects their quality and craftsmanship, a need that is frequently unmet by standard platforms.

Technical debt and performance bottlenecks further complicate the issue for fashion retailers. High-resolution imagery is essential for showcasing fabric details and designs, but poorly optimized systems often lead to slow page loads and high bounce rates. This friction in the user journey directly translates to lost sales and decreased customer loyalty. Retailers are often caught in a tug-of-war between maintaining a visually rich site and ensuring a fast, responsive user experience, struggling to find a technology stack that can handle both effectively.

Data silos and fragmented administrative workflows represent another critical bottleneck in current systems. Managing inventory across multiple categories, tracking orders through various stages, and maintaining a clear view of customer preferences often requires jumping between disconnected tools. This inefficiency not only increases the risk of human error—such as overselling stock or delaying shipments—but also prevents business owners from making data-driven decisions. The lack of a centralized "command center" hampers the operational agility needed to thrive in a fast-paced retail environment.

User trust and security are increasingly fragile in the age of frequent data breaches. Many small-to-medium fashion retailers lack the infrastructure to implement robust security protocols, leaving them vulnerable and their customers' data at risk. A single security lapse can permanently damage a brand's reputation, yet implementing enterprise-grade security is often seen as too complex or costly for niche boutiques. This creates a barrier to entry for brands that want to offer a premium, secure online shopping experience without compromising on their core business focus.

Finally, the lack of scalability in traditional e-commerce architectures restricts long-term growth. Many platforms are built on "monolithic" structures that are difficult to upgrade or integrate with emerging technologies like AI-driven personalization or native mobile apps. As a brand's needs evolve, these rigid systems become liabilities, forcing expensive and disruptive re-platforming projects. This technological myopia prevents fashion entrepreneurs from building a future-proof foundation that can grow alongside their brand's ambition and market reach.

Luxelle is born from the need to solve these multifaceted problems through a cohesive, "Luxury First" technical solution. By leveraging the MERN stack, it provides the high-performance engine needed to deliver high-resolution beauty without sacrificing speed. Its modular design addresses the administrative fragmentation by centralizing control, while its built-in security and scalability ensure that brand integrity is preserved as the business expands. Luxelle doesn't just offer an online storefront; it provides a comprehensive digital infrastructure tailored for the unique demands of the premium fashion world.

# Chapter 2:
# Proposed System

## 2.1 Existing System & Limitations

Traditional e-commerce implementations often rely on legacy CMS platforms or basic template engines that were not designed for the high-intensity visual demands of modern fashion. These systems frequently utilize server-side rendering for every interaction, leading to "page flickers" and forced reloads that disrupt the luxury feeling of the browsing experience. For a shopper, this feels clunky and disconnected, far removed from the smooth, attentive service they would receive in a physical boutique. This lack of fluidity is a primary cause of user frustration and high exit rates.

The administrative experience in existing systems is equally cumbersome. Many legacy platforms have interfaces that are difficult to navigate, required extensive training even for simple tasks like adding a new product category. Bulk updates, inventory tracking across variants, and order status management often involve manual data entry into multiple fields, which is not only slow but highly prone to error. This administrative weight prevents fashion teams from focusing on creative direction and brand growth, trapping them in a cycle of repetitive technical maintenance.

Performance-wise, generic systems struggle to handle the high-resolution image galleries required for luxury goods. Without advanced lazy loading, edge caching, or efficient data fetching, these sites become heavy and non-responsive, especially on mobile devices where a significant portion of fashion shopping occurs. A slow site isn't just a technical flaw; it's a branding failure. It communicates a lack of professionalism and care, undermining the premium positioning of the items being sold and driving away discerning customers who value their time and experience.

Security in traditional setups is often an afterthought or relies on third-party plugins that create additional vulnerabilities. Simple password storage, lack of structured session management, and unprotected API routes make these systems easy targets for malicious actors. Furthermore, the lack of a centralized authentication strategy means that user data is often scattered and difficult to manage securely. This "patchwork" approach to security is a significant risk factor for growing businesses that need to build and maintain trust with their high-value client base.

Integration and scalability are perhaps the greatest long-term limitations. Monolithic legacy systems make it incredibly difficult to add modern features like AI-powered style recommendations, dark mode toggles, or real-time notification systems. When a brand wants to innovate, they often find themselves held back by a rigid codebase that requires expensive custom development for even minor changes. This technological stagnation leaves them vulnerable to competitors who are more agile and able to adapt to changing user expectations and emerging retail trends.

The Luxelle platform is designed specifically to overcome these historic barriers. By moving to a decoupled, state-of-the-art architecture, it replaces clunky reloads with smooth transitions and turns difficult administration into an intuitive, productive experience. It optimizes asset delivery for lightning-fast speeds and prioritizes absolute security through modern token-based authentication. Most importantly, it is built to be a living system that can evolve, ensuring that a fashion brand's digital presence remains as cutting-edge as the collections it showcases.

## 2.2 Objectives

The primary objective of the Luxelle platform is to elevate the digital brand identity of fashion retailers. In the competitive online marketplace, first impressions are critical. Luxelle aims to provide a visually stunning interface that instantly communicates exclusivity, quality, and style. By moving away from standard, cookie-cutter e-commerce designs, the platform seeks to create a unique digital environment that resonates with fashion-conscious consumers, thereby strengthening brand loyalty and perceived value.

A key operational objective is to streamline the complex processes involved in inventory and order management. For administrators, the platform is designed to be a powerful tool that simplifies daily tasks such as updating stock levels, managing product variants like colors and sizes, and processing customer orders. By automating and organizing these workflows, Luxelle aims to reduce manual errors, save time, and allow business owners to operate more efficiently, effectively maximizing their productivity and operational throughput.

Enhancing user engagement and reducing cart abandonment are central to the platform's user-centric goals. Luxelle strives to create a frictionless user journey, from the moment a visitor lands on the homepage to the final confirmation of their purchase. This involves implementing intuitive navigation, smart search functionality, and a streamlined checkout process. By minimizing the steps required to find and buy products and by removing common friction points, the system encourages higher conversion rates and improves overall customer satisfaction. Ensuring scalability and performance is a fundamental architectural objective. As a business grows, its software infrastructure must be able to handle increased traffic, larger product catalogs, and more simultaneous transactions without degradation in speed or reliability. Luxelle is engineered to be robust and scalable, utilizing the non-blocking nature of Node.js and the horizontal scalability of MongoDB. The goal is to provide a platform that grows with the business, supporting its expansion from a boutique store to a large-scale retailer.

Another significant objective is to ensure the security and privacy of user data. With the increasing prevalence of cyber threats, building trust with customers is essential. Luxelle aims to implement rigorous security protocols, including secure authentication mechanisms and data encryption. The objective is to safeguard sensitive customer information, such as personal details and order history, ensuring compliance with privacy standards and fostering a secure environment for online transactions.

Lastly, the project aims to provide a data-driven foundation for business decision-making. By capturing detailed information on user interactions, order history, and product popularity, the system creates a valuable repository of data. This allows for future integration of advanced analytics and reporting tools. The objective is to empower business owners with insights that help them understand customer behavior, optimize their inventory, and tailor their marketing strategies for better results.

## 2.3 Hardware & Software Platforms

### Hardware Development Environment:

| | |
|---|---|
| **Minimum:** | • **Processor:** Dual-core Processor<br>• **RAM:** 4GB<br>• **Storage:** 256GB HDD |
| **Recommended:** | • **Processor**: Modern Multi-core Processor<br>• **RAM:** 8GB minimum recommended<br>• **Storage:** 512GB SSD for faster database I/O |

### Software Development Environment:

| | |
|---|---|
| **Operating System:** | Windows 10/11, macOS 10.15+, or Linux. |
| **Frontend** | Angular (v21.1.0), Tailwind CSS (v3.4.19) |
| **Backend** | Node.js with Express.js v5.1.0 |
| **Database:** | MongoDB (Local / Atlas) |
| **Code Editor:** | Visual Studio Code (Recommended). |
| **Browser:** | Chrome, Firefox, or Edge. |
| **Version Control:** | Git. |

## 2.4 Scope of Project

The scope of Luxelle encompasses a comprehensive Product Discovery module designed to facilitate an effortless shopping experience. This includes a dynamic and responsive product catalog that allows users to browse through high-resolution images and detailed descriptions. The system supports advanced filtering capabilities, enabling customers to narrow down their search by category, price range, brand, and other attributes. Additionally, a smart search function is included to help users quickly locate specific items, ensuring that the vast inventory is easily accessible and navigable.

User Management is another critical component within current scope. The system provides a secure and user-friendly registration and login process, featuring form validation and password encryption. Once authenticated, users have access to a personalized dashboard where they can manage their profiles, update their personal information, and maintain an address book for faster checkouts. The scope also includes a "Wishlist" feature, allowing users to save items for future consideration, which enhances user retention and engagement.

Transaction Processing forms the core of the commercial functionality. The project scope covers the entire lifecycle of a purchase, starting from a persistent shopping cart that retains items across sessions to a multi-step checkout workflow. This workflow collects necessary shipping and billing information, allows for the selection of payment methods, and culminates in a final order confirmation. Furthermore, the system tracks the order history for each user, providing them with transparency regarding their past purchases and current order status.

On the administrative side, the scope includes a powerful Admin Management Dashboard. This restricted area provides authorized personnel with full control over the platform's content and operations. Admins can perform CRUD (Create, Read, Update, Delete) operations on products, managing details such as pricing, stock levels, and imagery. The dashboard also facilitates User Management, allowing admins to view registered users, and provides tools for Order Management, including the ability to update order statuses (e.g., from "Processing" to "Shipped") and view detailed order summaries.

The security scope involves implementing robust measures to protect the platform and its users. This includes the implementation of JSON Web Token (JWT) based authentication for stateless and secure session management. It also covers the protection of API routes to ensure that sensitive data and administrative functions are only accessible to authorized users. Input validation on both the client and server sides is included to prevent common web vulnerabilities such as SQL injection (or NoSQL injection in this context) and Cross-Site Scripting (XSS).

Finally, the technical scope defines the boundaries of the platform's operation. Luxelle is designed as a web-based application accessible via standard web browsers on desktop and mobile devices. It requires an active internet connection to function, as it relies on a cloud-hosted database and backend server. The current scope focuses on digital payments via simulation (or integration placeholders) and standard delivery workflows.

## 2.5 Feasibility Study

The Technical Feasibility of Luxelle is highly rated due to the maturity and performance of the chosen MERN stack. Node.js provides a high-performance, non-blocking runtime that is ideal for handling the asynchronous nature of e-commerce transactions and API requests. Angular allows for the construction of a modular, typesafe frontend that is both maintainable and highly interactive. MongoDB's flexible schema is perfect for an evolving inventory where products might have different attributes. The widespread availability of documentation and community support for these technologies ensures that any technical challenges encountered during development can be efficiently resolved.

Economically, the project is highly feasible as it primarily leverages open-source technologies. By avoiding expensive licensing fees associated with proprietary platforms like SAP or Shopify Plus, Luxelle reduces the initial capital expenditure significantly. The development tools, including VS Code, Git, and the Node ecosystem, are freely available, and the stack can be deployed on cost-effective cloud infrastructure like AWS or DigitalOcean. The primary investment is in development time rather than software licenses, making it an attractive model for both startups and established brands looking to own their infrastructure.

Operational Feasibility is ensured through Luxelle's focus on user-centric design for both customers and administrators. For the end-user, the platform follows established e-commerce patterns while elevating the aesthetic, ensuring that the interface is familiar yet premium. For the admin, the dashboard is designed to simplify complex data entry, reducing the training required to manage the store. The system's ability to automate inventory updates and provide clear order tracking means that the operational burden on the staff is lowered, allowing them to scale their business without a linear increase in administrative work.

Market Feasibility is supported by the growing consumer preference for premium, curated online shopping experiences. As high-end fashion brands continue to transition their sales online, there is a clear demand for platforms that can preserve brand prestige in a digital format. Luxelle's "Luxury First" philosophy directly addresses this market gap, offering a solution that is more sophisticated than generic templates but more affordable than completely custom enterprise builds. The platform's ability to handle high-resolution imagery and smooth transitions aligns perfectly with the expectations of fashion-conscious shoppers.

Legal and Security Feasibility is addressed through the implementation of industry-standard protocols. The use of JWT for authentication and Bcrypt for password hashing ensures that user data is handled in a way that respects privacy and meets basic data protection regulations. By centralizing data in a controlled MongoDB environment and securing API endpoints, the platform minimizes the surface area for legal liabilities. Compliance with web accessibility standards can also be integrated into the Angular components, ensuring that the platform remains legally compliant and inclusive to all users as it grows.

Finally, the Schedule Feasibility is managed through a modular development approach. By breaking the system down into independent feature sets—Authentication, Product Discovery, Cart, and Admin—the project can be developed and tested in phases. This iterative approach allows for the core functionality to be established quickly, with more complex features integrated as the project progresses. This structure ensures that a functional version of the platform can be delivered within a reasonable timeframe, providing value to the business and stakeholders without the delays often associated with monolithic software projects.

# Chapter 3:
# System Design

## 3.1 System Architecture

The architecture of Luxelle is built on the principle of decoupling, separating the user interface from the business logic and data storage. At the core is the Single Page Application (SPA) model powered by Angular. This ensures that the user never experiences full-page reloads; instead, content is fetched dynamically via the REST API and rendered instantly on the client side. This horizontal separation allows for independent scaling of the frontend and backend, ensuring that high traffic on the storefront doesn't directly overwhelm the administrative or database processes in a way that would crash the entire system.

On the client side, Luxelle utilizes Angular’s Signal-based state management. This modern approach allows for fine-grained reactivity, where only the specific parts of the UI that need to change—such as a cart counter or a price filter—are updated. This reduces the computational overhead on the user’s device and creates an "instant" feel that is critical for a luxury brand. The use of standalone components and lazy-loaded routes ensures that the initial bundle size is kept small, allowing the site to load quickly even on slower mobile connections, which is where real-world e-commerce often happens.

The backend is a RESTful API built with Node.js and Express.js. This layer acts as the orchestrator of the system, verifying user permissions, processing business logic, and communicating with the database. Because the backend is stateless, it can easily handle peak traffic by running multiple instances behind a load balancer. Every request is verified using JSON Web Tokens (JWT), ensuring that the server doesn't need to maintain a memory-heavy session table, further improving performance and security. This lean backend architecture is the foundation of Luxelle’s high throughput and low latency.

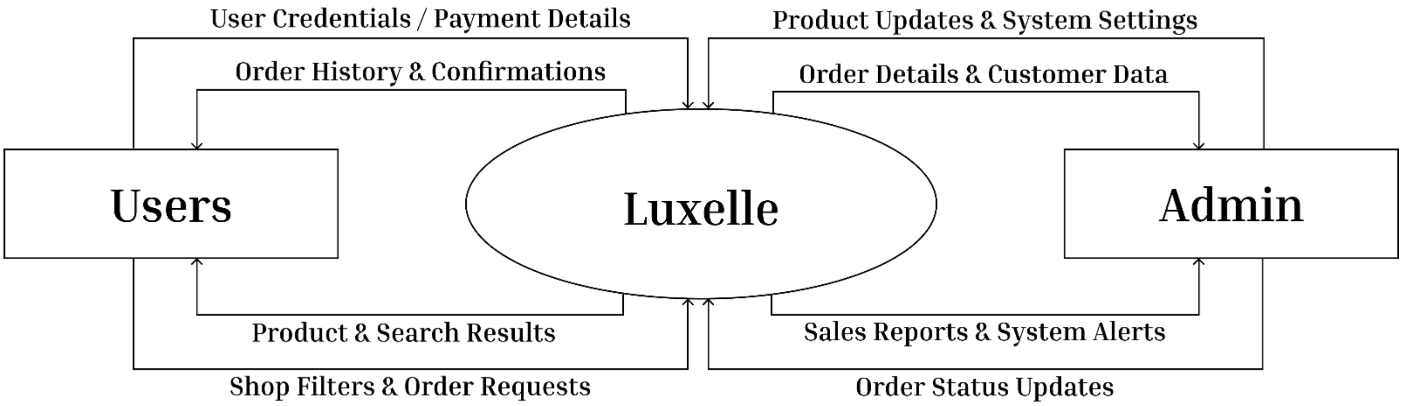
Data persistence is handled by MongoDB, a NoSQL database that offers horizontal scalability and a flexible schema. Unlike traditional relational databases that require strict tables and joins, MongoDB stores data in documents that match the structure of our JavaScript objects. This makes data retrieval faster and development more intuitive. For e-commerce, this is particularly valuable because it allows us to store complex, nested objects—like a structured shipping address or a variable array of product colors—directly within a single record.

Communication between these layers happens exclusively over HTTPS using JSON payloads. This standardized communication means that the Luxelle backend could theoretically power not just our web application, but also future iOS, Android, or even smart-watch applications without needing to be rewritten. This universal API design is a core component of Luxelle's future-proofing strategy. It ensures that the brand's data is the "source of truth" accessible across any digital touchpoint the customer chooses to use, providing a truly omni-channel experience.

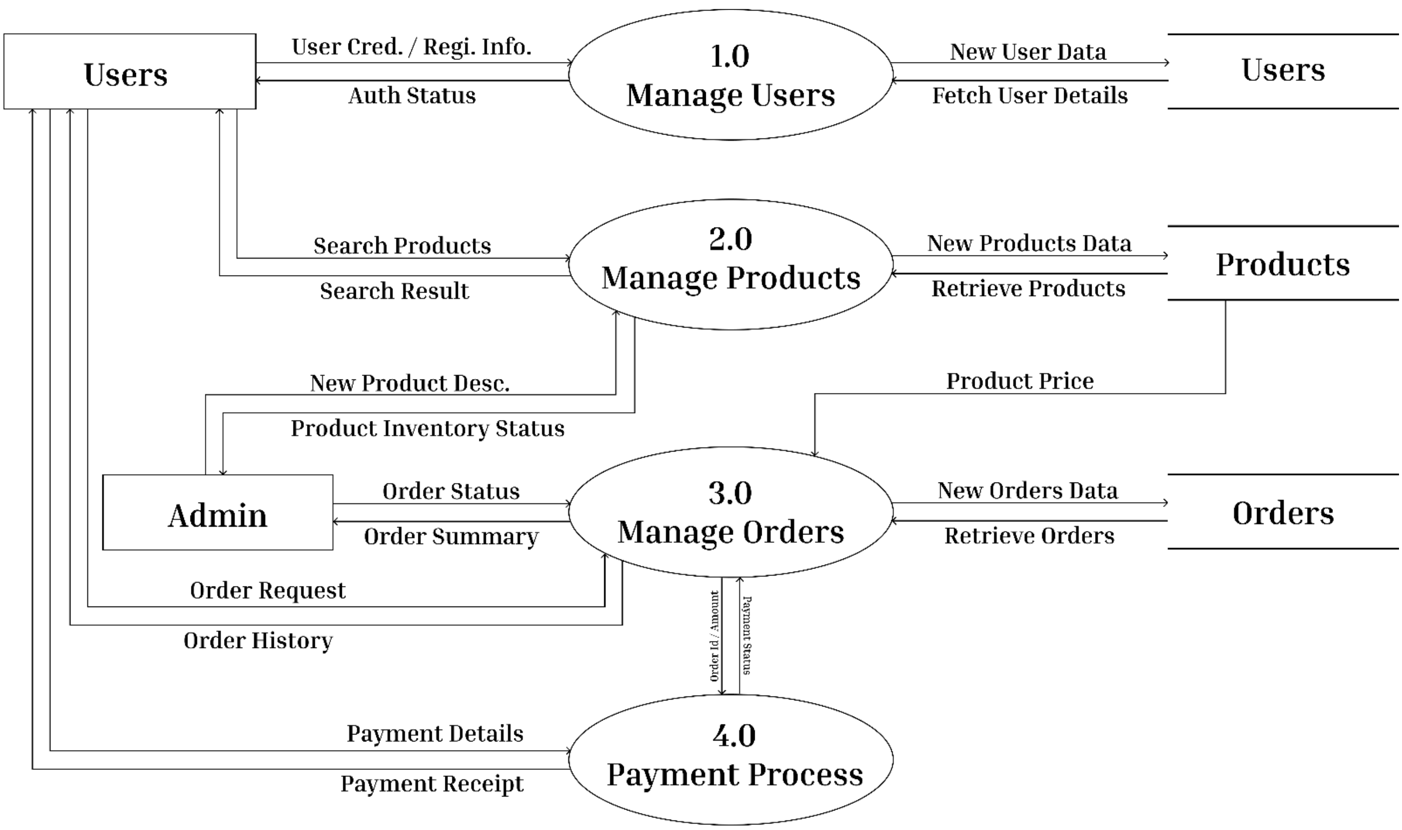
Finally, the architecture includes a comprehensive Security and Middleware layer. All incoming data is sanitized and validated against Mongoose schemas before it ever reaches the database, preventing injection attacks. CORS policies are strictly defined to ensure that only our trusted frontend can communicate with the backend. This multi-layered defense strategy—from the client-side guards to the server-side validation—ensures that Luxelle is not just fast and beautiful, but also a fortress for user data and brand integrity.

# 3.2 DFD (Data Flow Diagram)

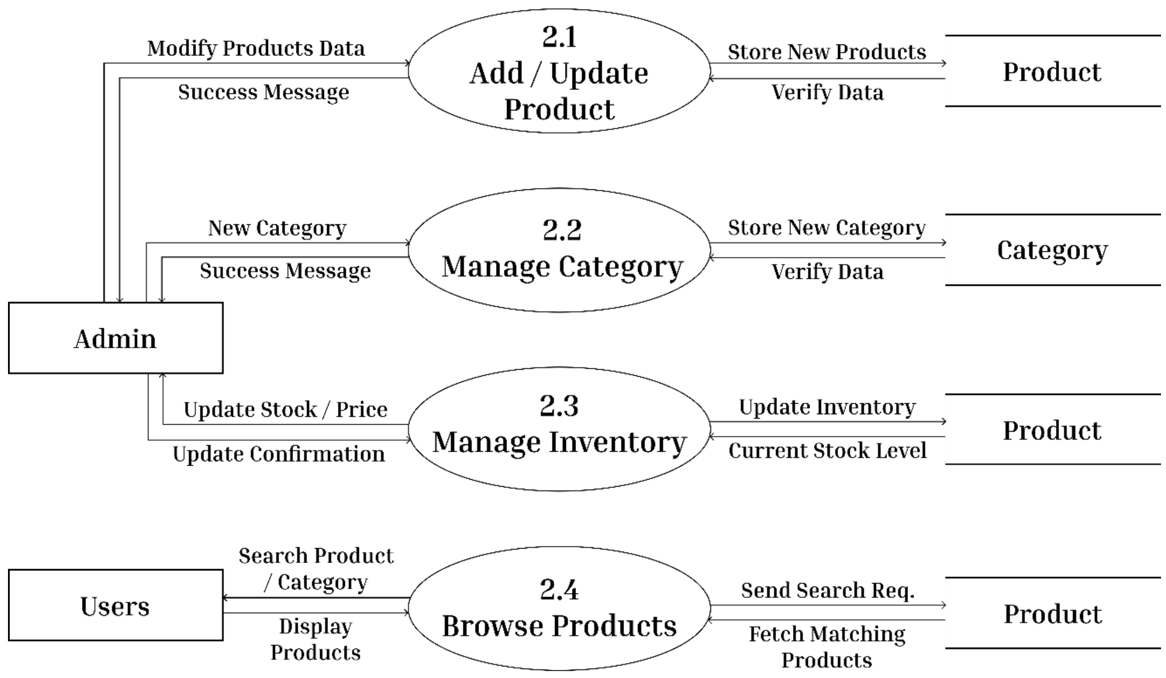
## DFD Level 0:



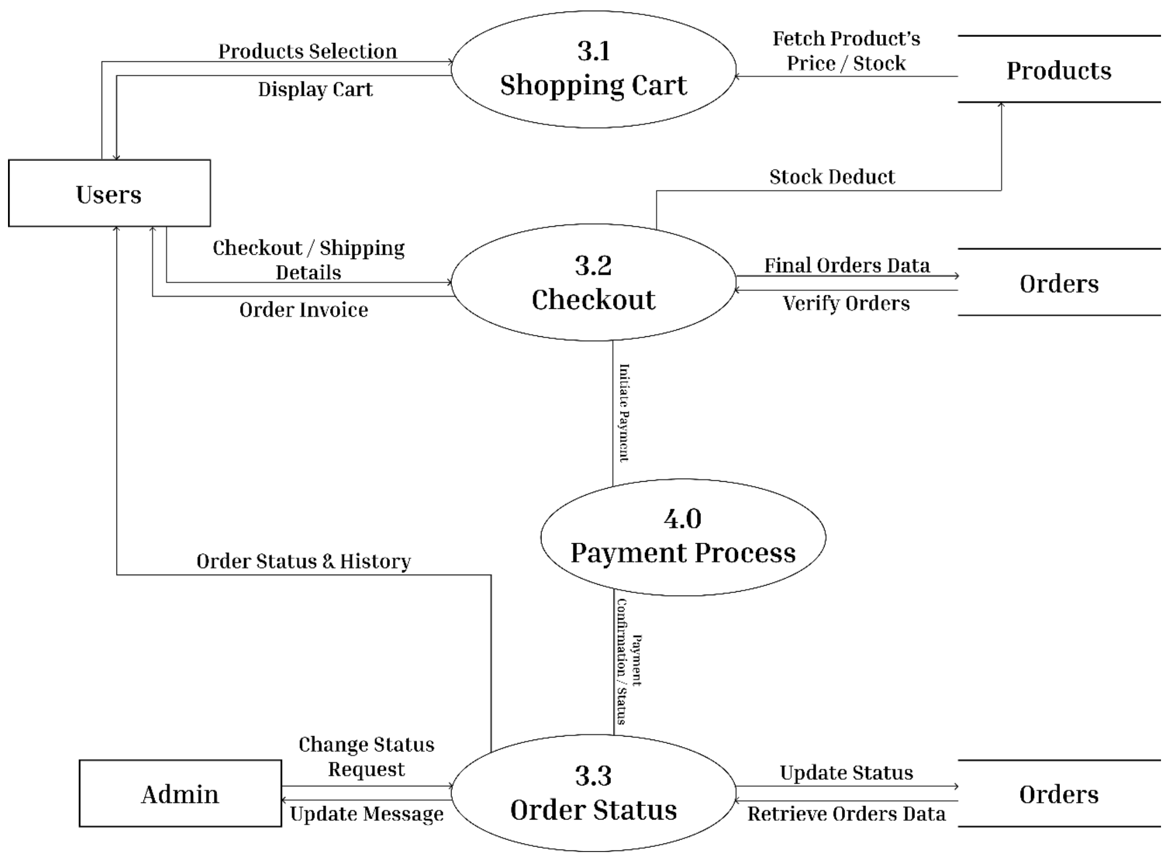

## DFD Level 1:



# DFD Level 2.1:

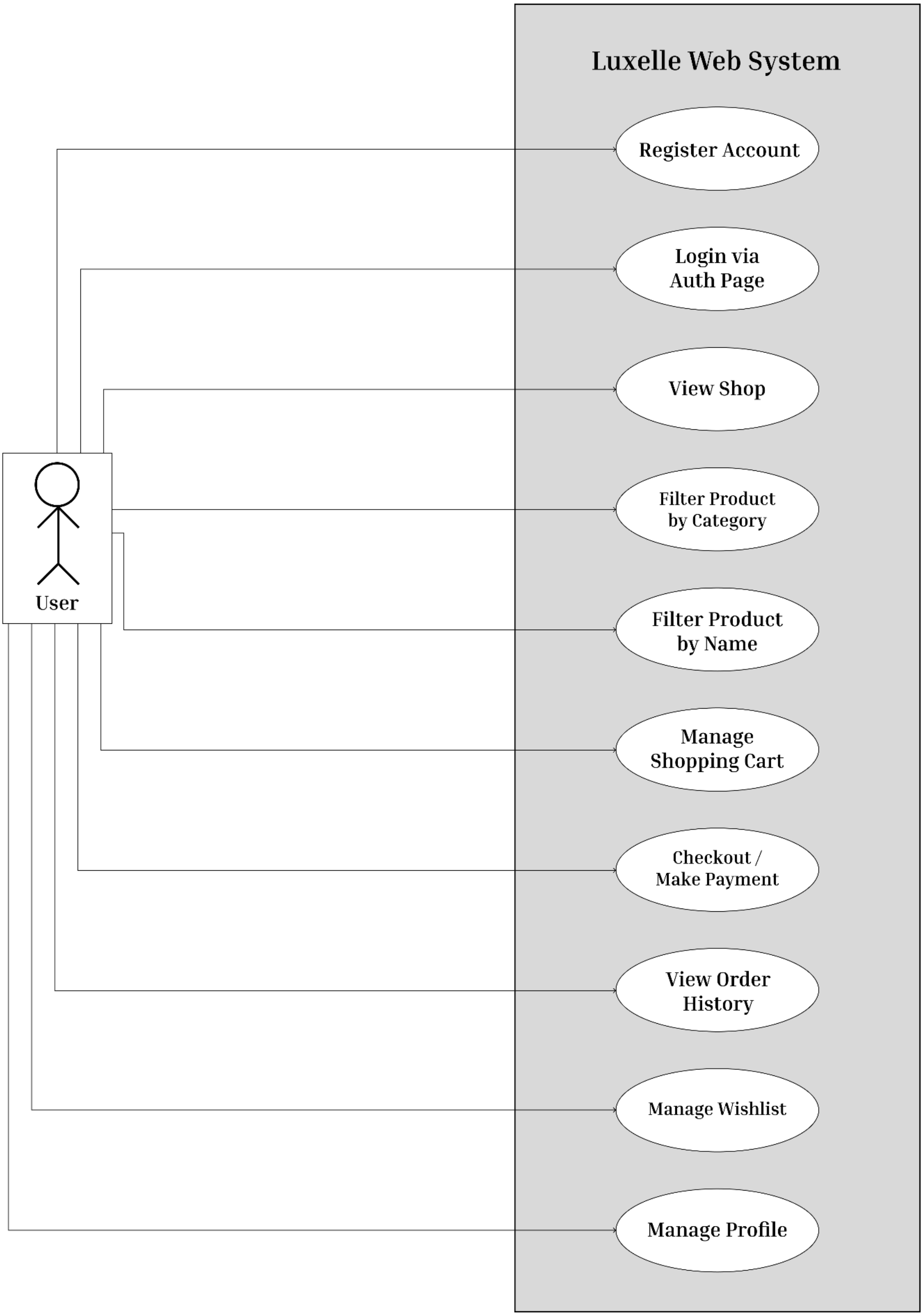



# DFD Level 3.1:

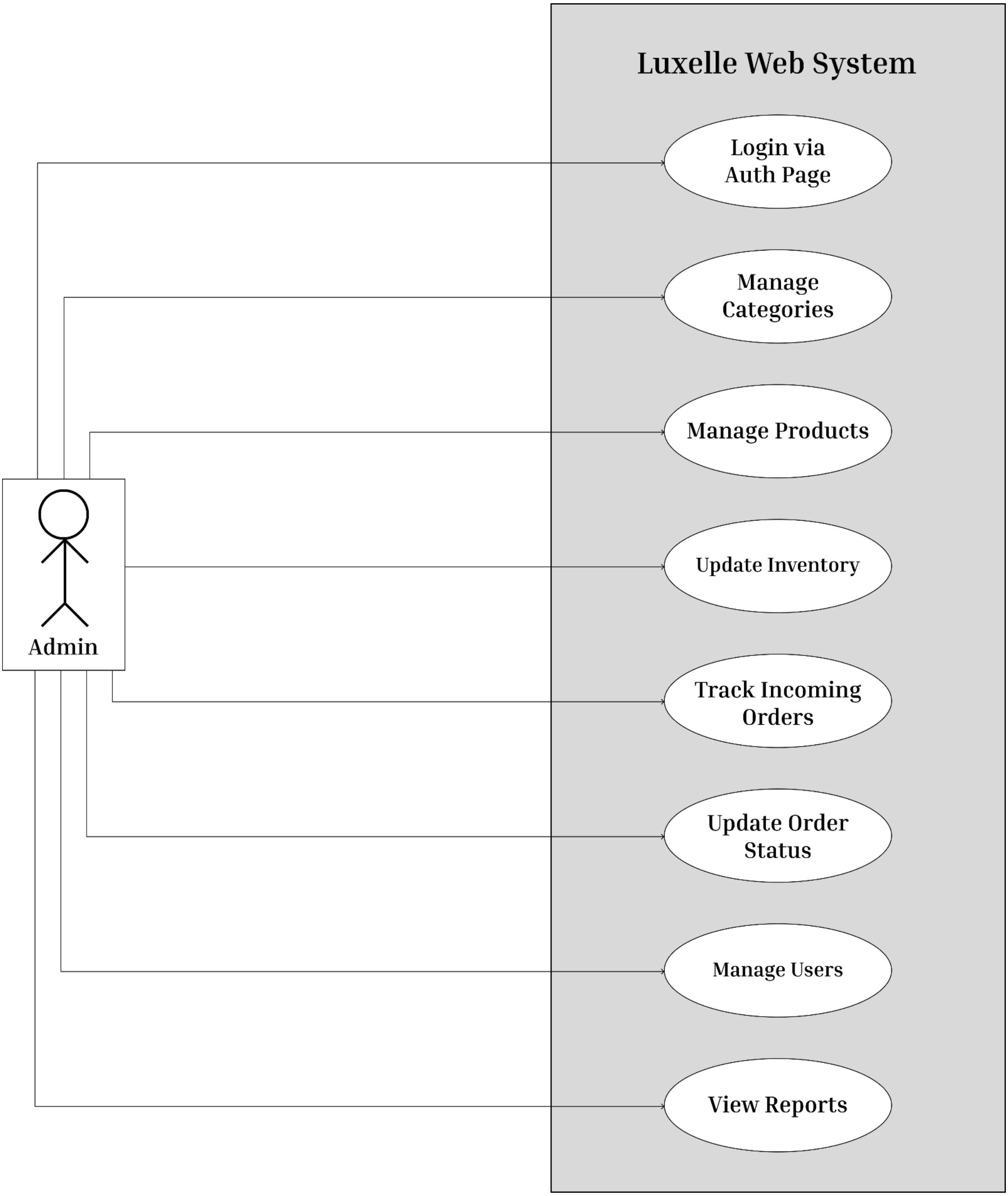


# 3.3 UML Diagram:

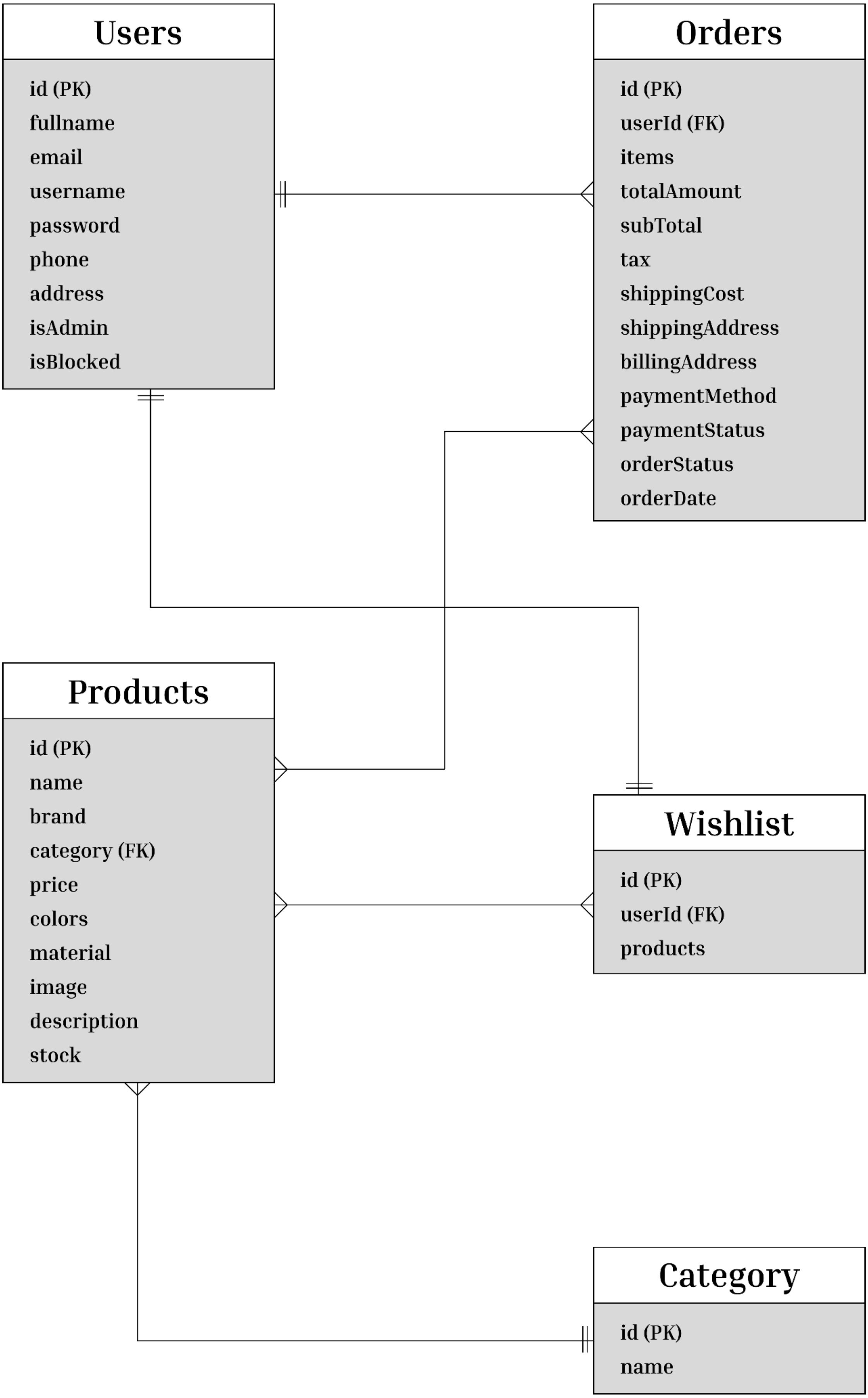
## Use Case Diagram (Users):



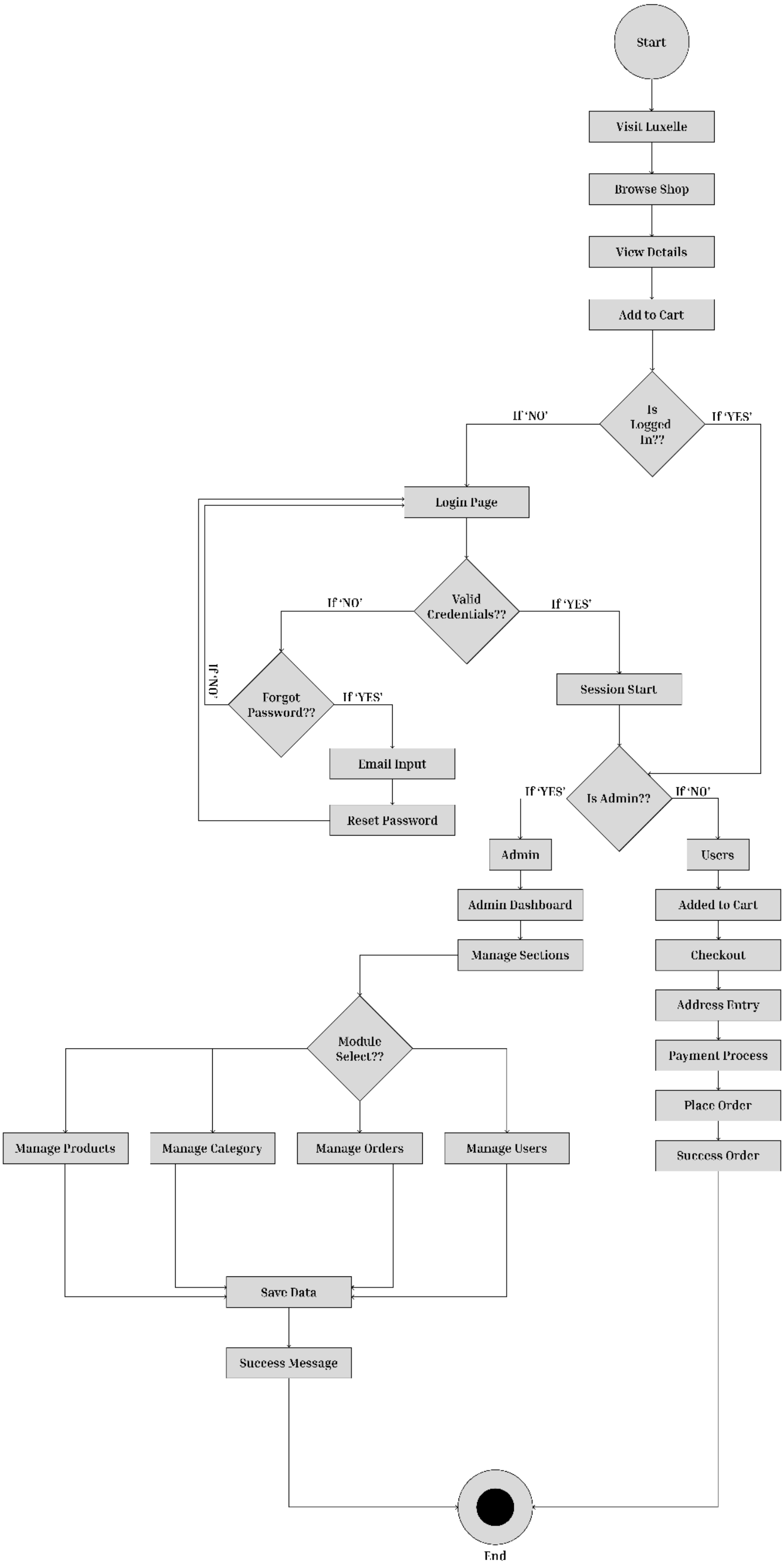
## Use Case Diagram (Admin):



## 3.3.2 ER Diagram



# 3.3.3 Activity Diagram



## 3.4 Data Dictionary

The Data Dictionary serves as a centralized repository of information about data, encompassing its meaning, relationships to other data, origin, usage, and format. In the context of the Luxelle platform, it provides a structured and comprehensive breakdown of the core database collections implemented in MongoDB.

**1). Users Table:**

| Field | Type | Constraint | Description |
|---|---|---|---|
| **_id** | ObjectId | Primary Key | A unique identifier automatically generated by MongoDB for each user record. It serves as the primary key for referencing users across the system. |
| **fullname** | String | Required | The full name of the registered user, used for identification, personalization, and communication purposes. |
| **email** | String | Unique, Required | The user's primary email address, used for authentication, communication, and account recovery. Must be unique. |
| **username** | String | Unique, Required | A unique login handle chosen by the user for authentication and account distinction. |
| **password** | String | Required | A securely hashed representation of the user’s password, ensuring protection against unauthorized access. |
| **phone** | String | Optional | The user’s contact number, optionally provided for delivery updates, verification, or security measures. |
| **address** | Object | Nested | A structured object containing the user's address details, including street, city, state, zip code, and country. |
| **isAdmin** | Boolean | Default: false | A flag indicating whether the user has administrative |

## 2. Products Table:

| Field | Type | Constraint | Description |
|---|---|---|---|
| **_id** | ObjectId | Primary Key | A unique identifier automatically generated by MongoDB for each product record. |
| **name** | String | Required | The customer-facing name of the product displayed within the catalog. |
| **brand** | String | Required | The brand or manufacturer associated with the product, used for filtering and categorization. |
| **category** | String | Enum | Specifies the product classification. Allowed values include Bags, Watches, Jewelry, Sunglasses, and Belts. |
| **price** | Number | Required | The retail price of the product used for billing and financial calculations. |
| **colors** | Array | Optional | A list of available color variants for the product. |
| **material** | String | Optional | Describes the primary material(s) used in the product's construction. |
| **image** | String | Required | A URL or file path referencing the product's primary image. |
| **description** | String | Optional | A detailed explanation of the product's features, benefits, and specifications. |
| **stock** | Number | Default: 0 | Indicates the available inventory quantity of the product. |

## 3. Orders Table:

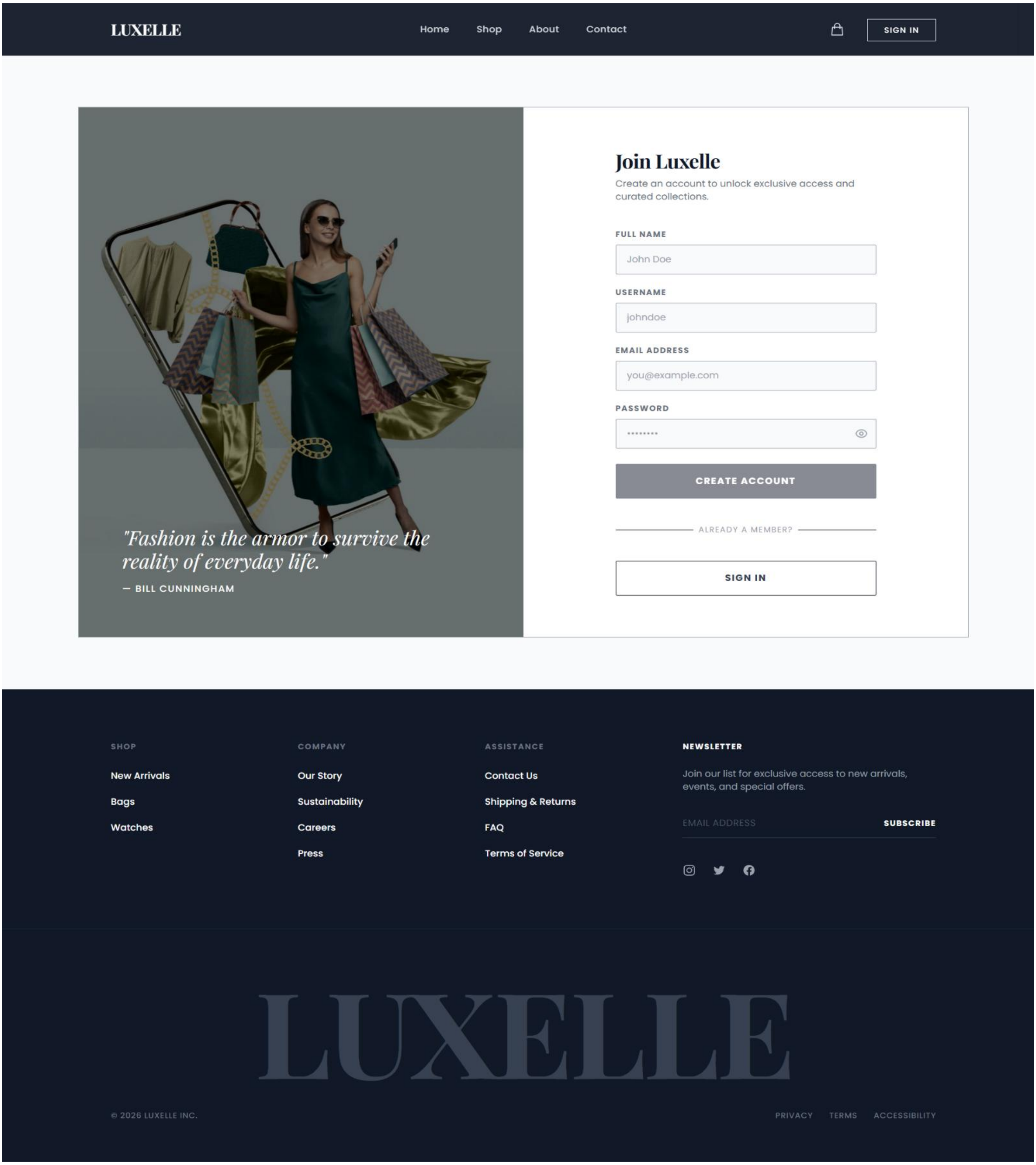
| Field | Type | Constraint | Description |
|---|---|---|---|
| **_id** | ObjectId | Primary Key | A unique identifier automatically generated by MongoDB for each order. |
| **user** | ObjectId | Ref: Users | A reference linking the order to the user who placed it. |
| **items** | Array | Required | A collection of purchased products including quantity, selected variants, and pricing snapshot. |
| **totalAmount** | Number | Required | The final payable amount, including subtotal, tax, and shipping charges. |
| **subtotal** | Number | Default: 0 | The total cost of items before applying tax and shipping fees. |
| **tax** | Number | Default: 0 | The calculated tax applied to the order. |
| **shippingCost** | Number | Default: 0 | The delivery charge associated with the order. |
| **shippingAddress** | Object | Required | The destination address where the order will be delivered. |
| **billingAddress** | Object | Required | The address associated with the payment method used for the order. |
| **paymentMethod** | String | Enum | Specifies the payment option. Allowed values: Card, UPI, COD. |
| **paymentStatus** | String | Default: Pending | Tracks payment progress (e.g., Pending, Completed, Failed). |
| **orderStatus** | String | Default: Confirmed | Tracks order lifecycle (Confirmed, Processing, Shipped, Delivered, Cancelled). |

## 4. Wishlist Table:

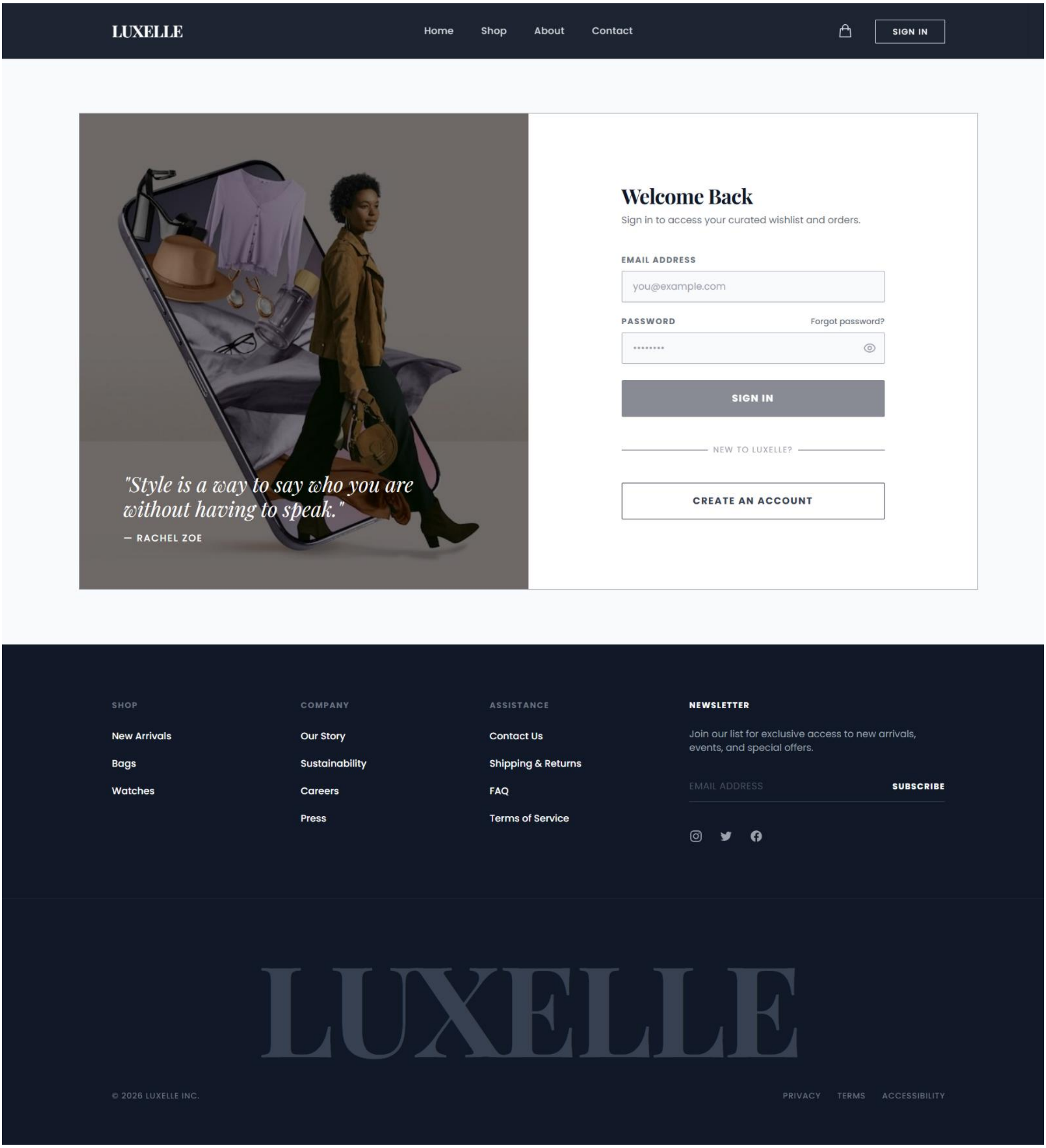
| Field | Type | Constraint | Description |
|---|---|---|---|
| **_id** | ObjectId | Primary Key | A unique identifier automatically generated by MongoDB for each wishlist record. |
| **user** | ObjectId | Ref: Users | A reference linking the wishlist to the specific user who owns it. Each wishlist is associated with a single user. |
| **products** | Array | Ref: Products | A dynamic collection of product references representing items saved by the user for future consideration. |
| **createdAt** | Date | Default: Date.now | The timestamp indicating when the wishlist was initially created. |
| **updatedAt** | Date | Auto-updated | The timestamp reflecting the most recent modification made to the wishlist (e.g., product added or removed). |

# 3.5 Interface (Screenshots)

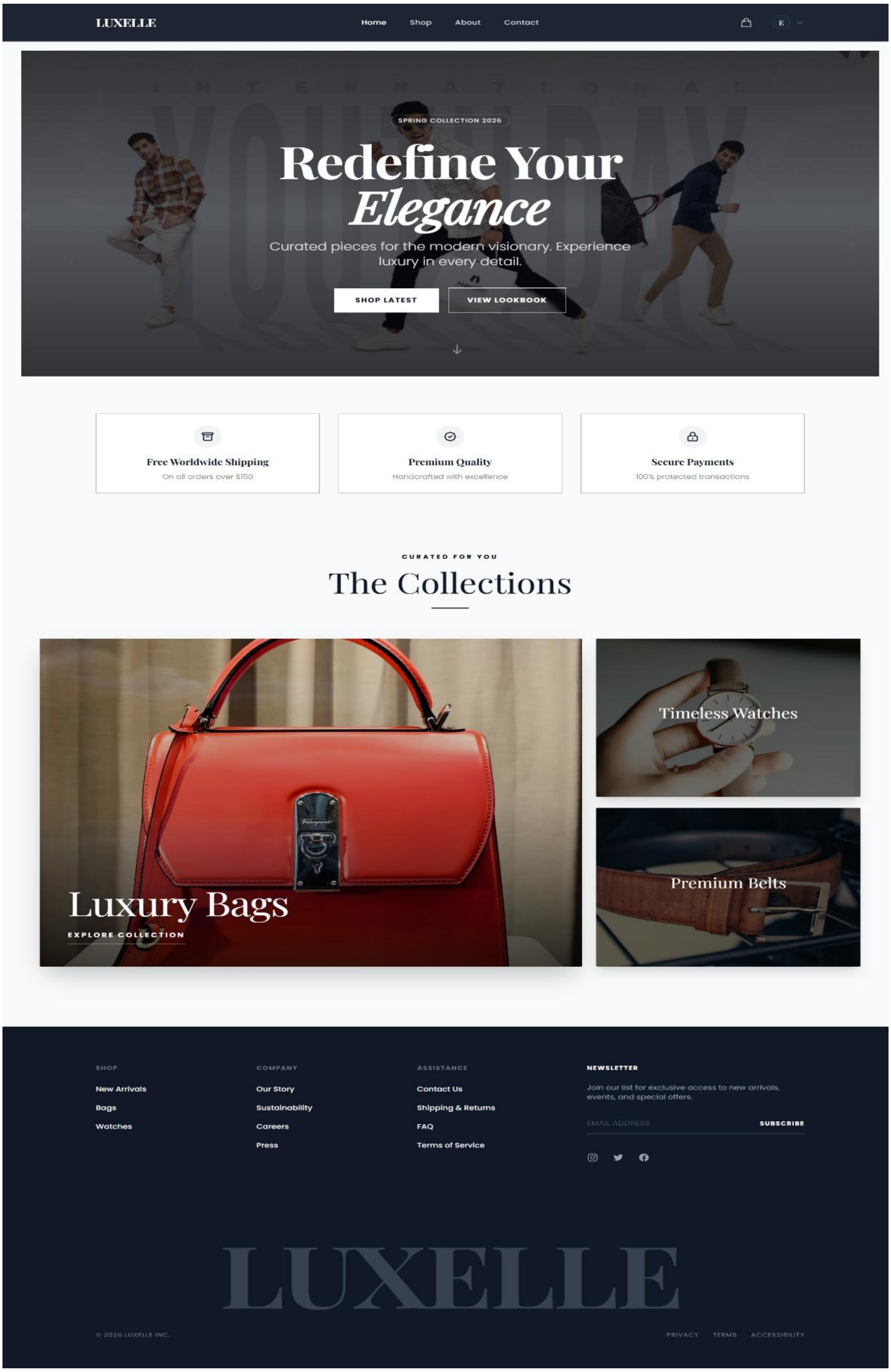
## Registration Page:



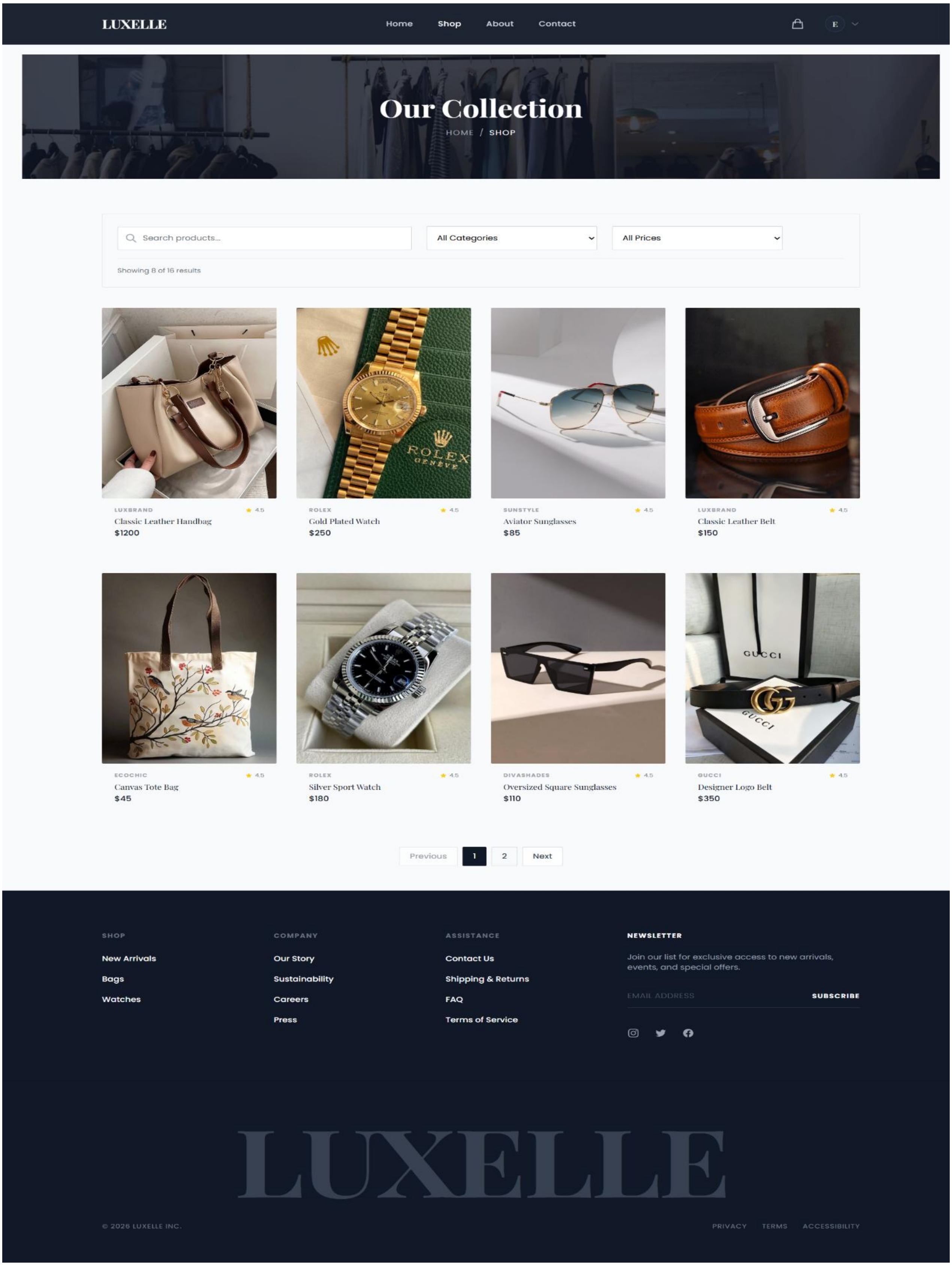
## Login Page:


LUXELLE
Home
Shop
About
Contact
SIGN IN
"Style is a way to say who you are without having to speak."
— RACHEL ZOE
Welcome Back
Sign in to access your curated wishlist and orders.
EMAIL ADDRESS
you@example.com
PASSWORD
Forgot password?
SIGN IN
NEW TO LUXELLE?
CREATE AN ACCOUNT
SHOP
New Arrivals
Bags
Watches
COMPANY
Our Story
Sustainability
Careers
Press
ASSISTANCE
Contact Us
Shipping & Returns
FAQ
Terms of Service
NEWSLETTER
Join our list for exclusive access to new arrivals, events, and special offers.
EMAIL ADDRESS
SUBSCRIBE
LUXELLE
© 2026 LUXELLE INC.
PRIVACY
TERMS
ACCESSIBILITY

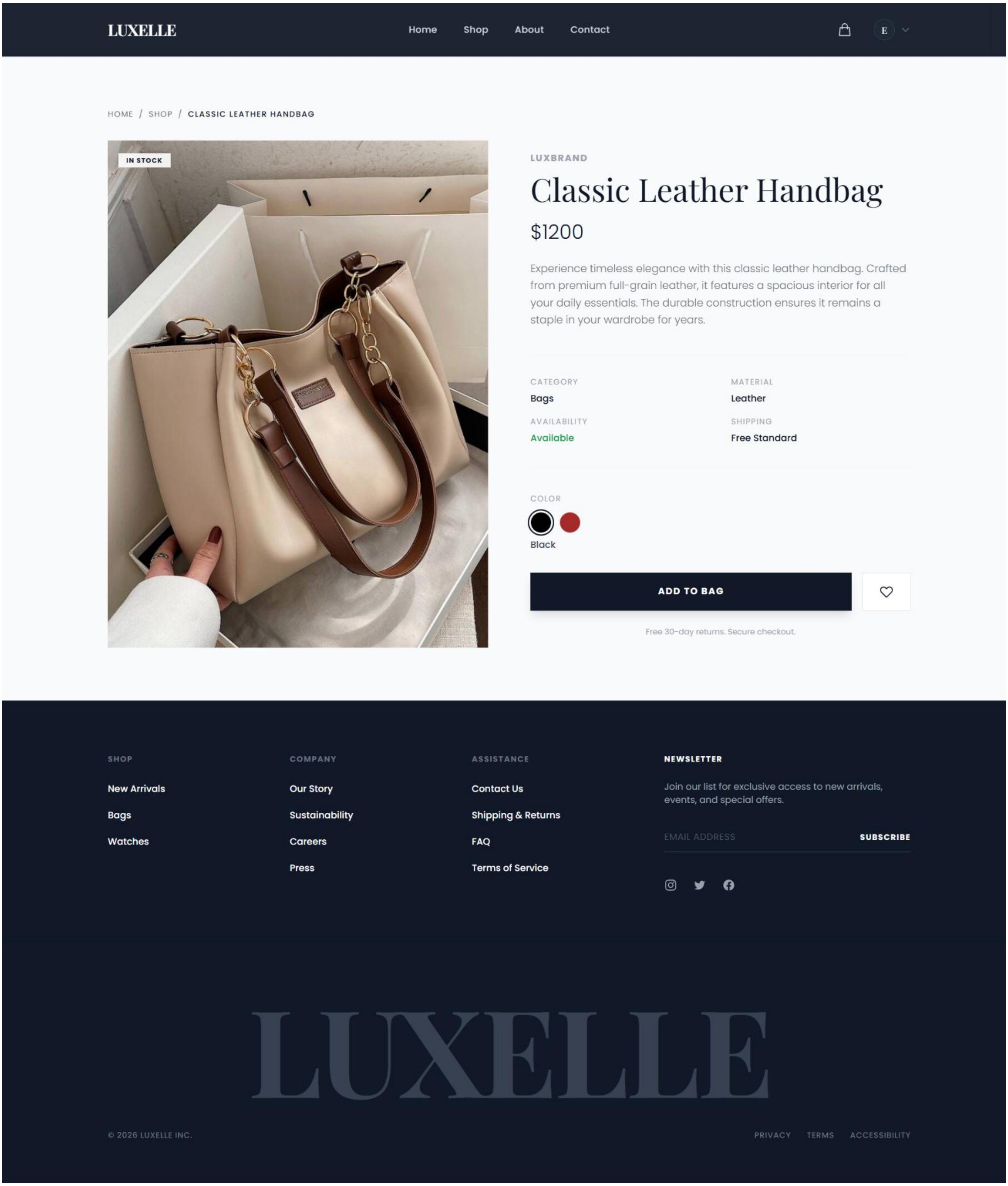
## Home Page:


LUXELLE
Home
Shop
About
Contact
SPRING COLLECTION 2026
Redefine Your
Elegance
Curated pieces for the modern visionary. Experience luxury in every detail.
SHOP LATEST
VIEW LOOKBOOK
Free Worldwide Shipping
On all orders over $150
Premium Quality
Handcrafted with excellence
Secure Payments
100% protected transactions
CURATED FOR YOU
The Collections
Luxury Bags
EXPLORE COLLECTION
Timeless Watches
Premium Belts
SHOP
New Arrivals
Bags
Watches
COMPANY
Our Story
Sustainability
Careers
Press
ASSISTANCE
Contact Us
Shipping & Returns
FAQ
Terms of Service
NEWSLETTER
Join our list for exclusive access to new arrivals, events, and special offers.
EMAIL ADDRESS
SUBSCRIBE
LUXELLE
© 2026 LUXELLE INC.
PRIVACY
TERMS
ACCESSIBILITY

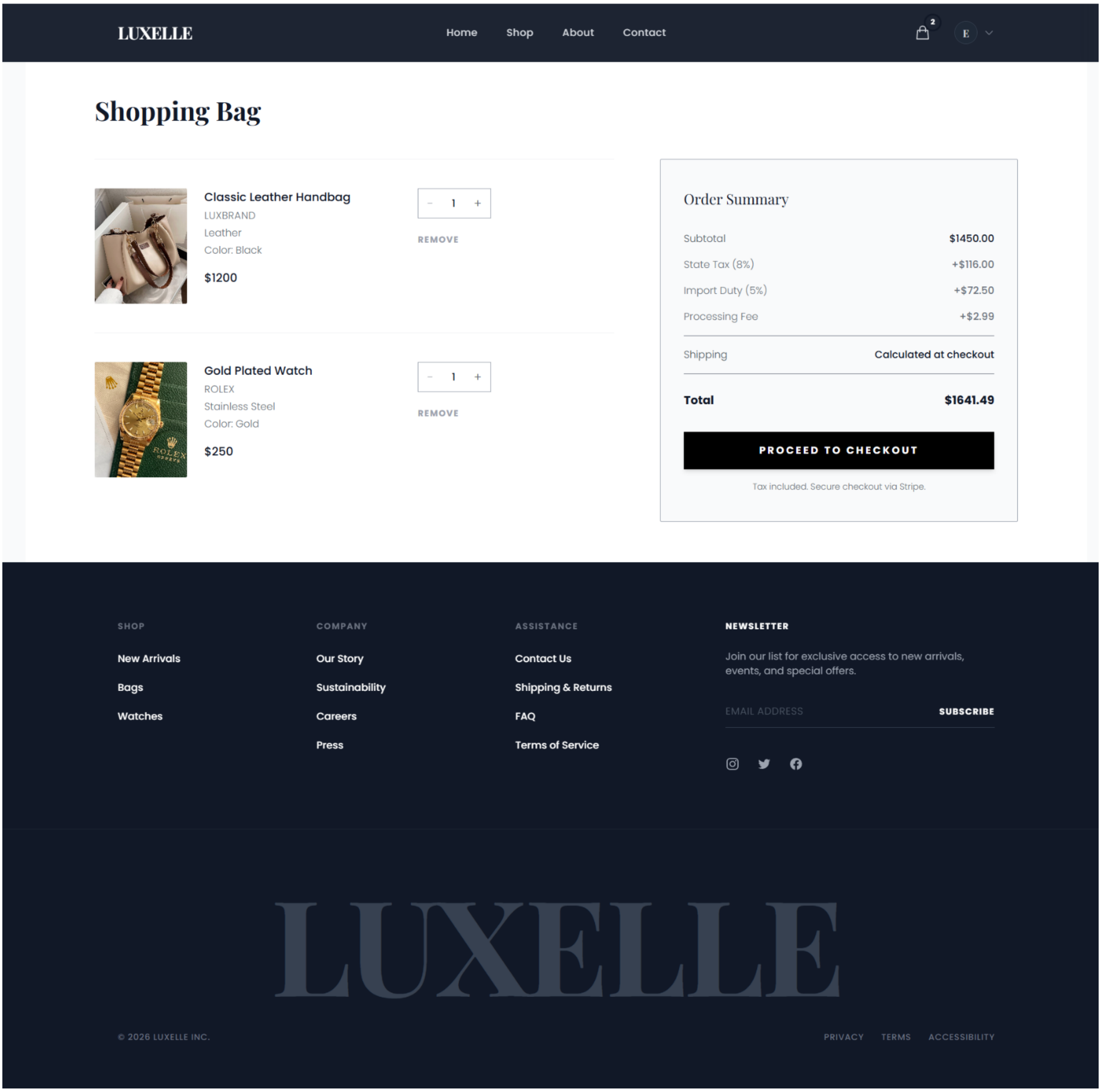
## Shop Page:


LUXELLE
Home
Shop
About
Contact
Our Collection
HOME / SHOP
Search products...
All Categories
All Prices
Showing 8 of 16 results
LUXBRAND
Classic Leather Handbag
$1200
ROLEX
Gold Plated Watch
$250
SUNSTYLE
Aviator Sunglasses
$85
LUXBRAND
Classic Leather Belt
$150
ECOCHIC
Canvas Tote Bag
$45
ROLEX
Silver Sport Watch
$180
DIVASHADES
Oversized Square Sunglasses
$110
GUCCI
Designer Logo Belt
$350
Previous
1
2
Next
SHOP
New Arrivals
Bags
Watches
COMPANY
Our Story
Sustainability
Careers
Press
ASSISTANCE
Contact Us
Shipping & Returns
FAQ
Terms of Service
NEWSLETTER
Join our list for exclusive access to new arrivals, events, and special offers.
EMAIL ADDRESS
SUBSCRIBE
LUXELLE
© 2026 LUXELLE INC.
PRIVACY
TERMS
ACCESSIBILITY

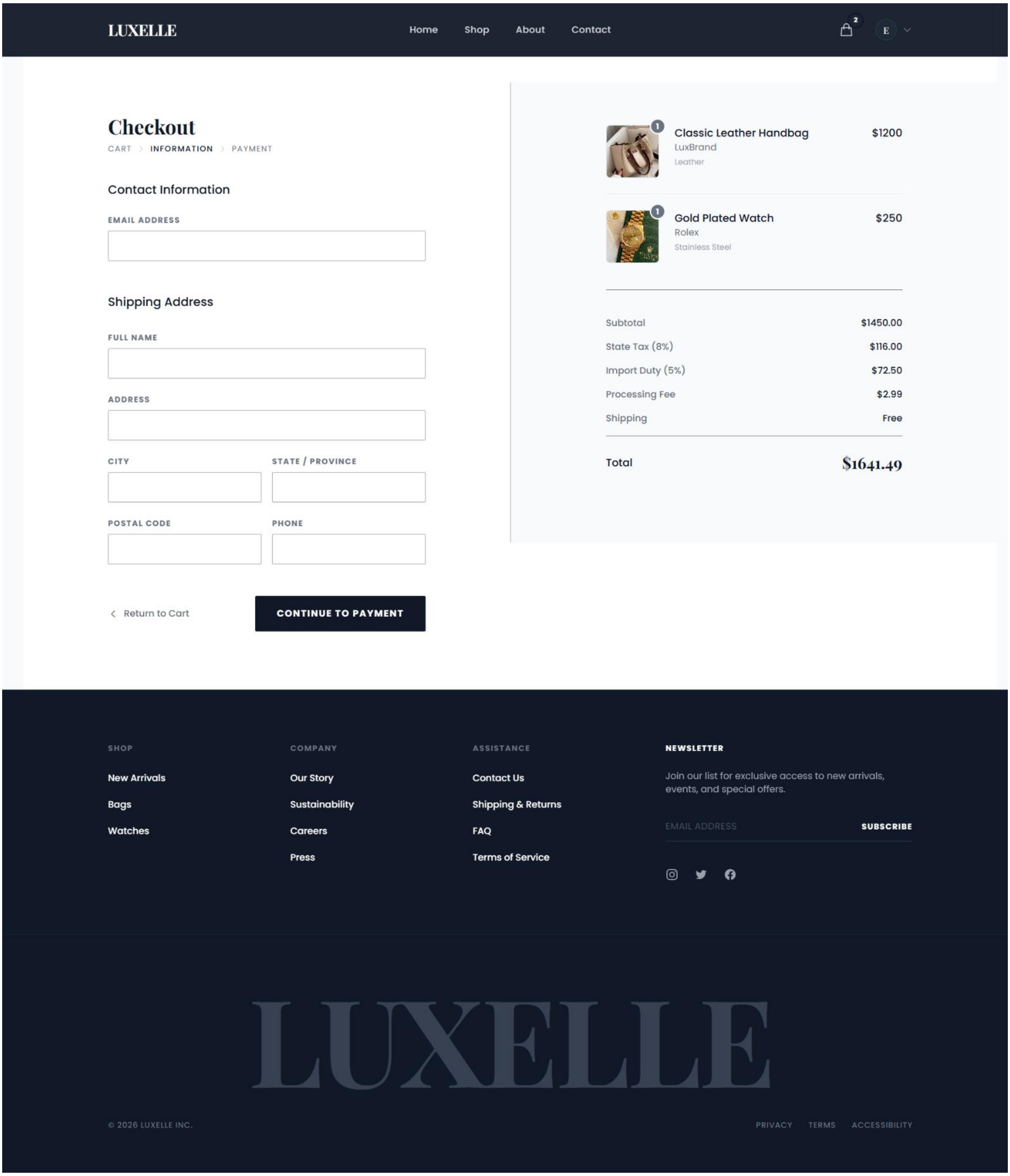
# Details Page:



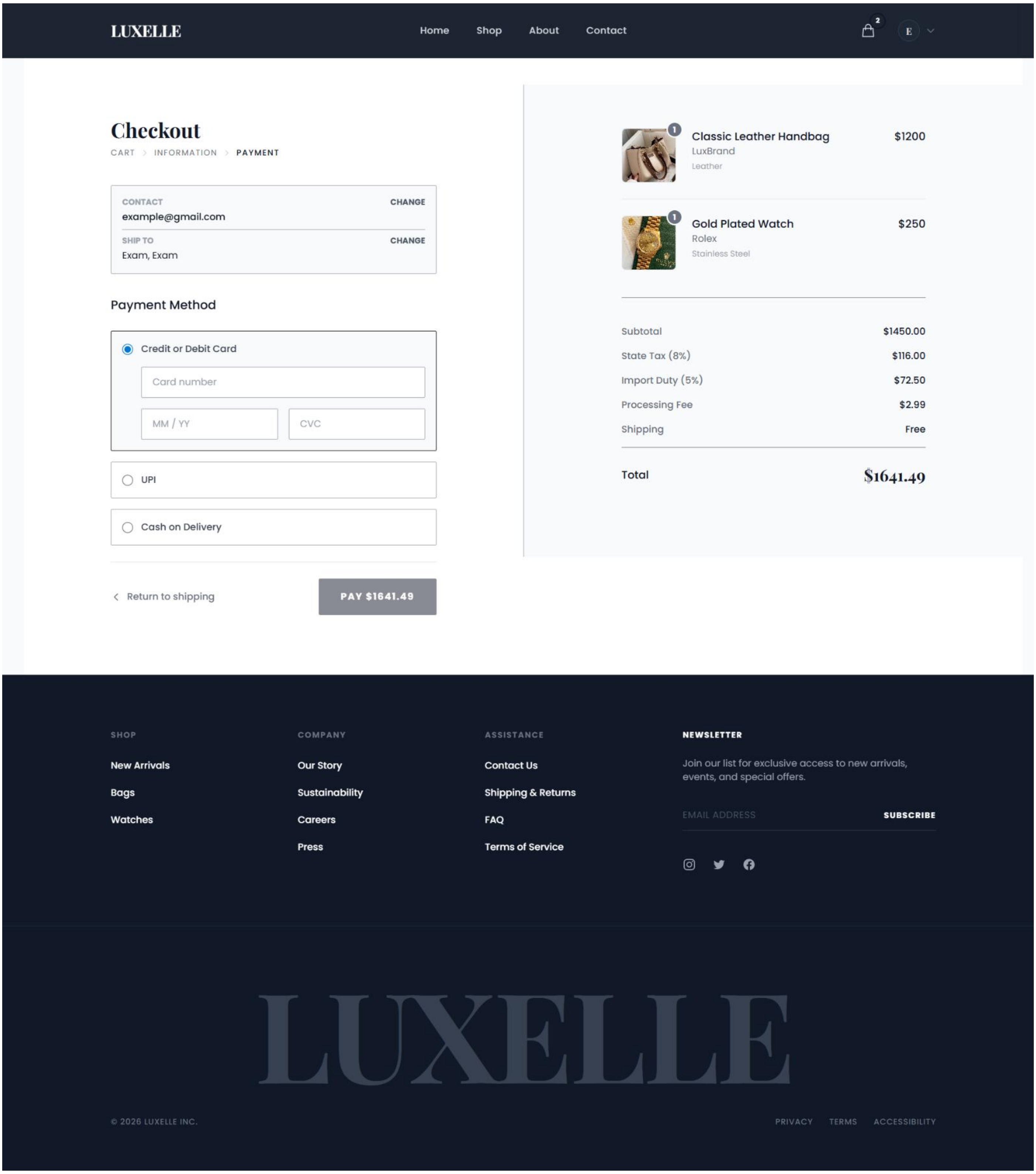
# Cart Page:


LUXELLE
Home
Shop
About
Contact
2
E
Shopping Bag
Classic Leather Handbag
LUXBRAND
Leather
Color: Black
$1200
1
REMOVE
Gold Plated Watch
ROLEX
Stainless Steel
Color: Gold
$250
1
REMOVE
Order Summary
Subtotal $1450.00
State Tax (8%) +$116.00
Import Duty (5%) +$72.50
Processing Fee +$2.99
Shipping Calculated at checkout
Total $1641.49
PROCEED TO CHECKOUT
Tax included. Secure checkout via Stripe.
SHOP
New Arrivals
Bags
Watches
COMPANY
Our Story
Sustainability
Careers
Press
ASSISTANCE
Contact Us
Shipping & Returns
FAQ
Terms of Service
NEWSLETTER
Join our list for exclusive access to new arrivals, events, and special offers.
EMAIL ADDRESS
SUBSCRIBE
LUXELLE
© 2026 LUXELLE INC.
PRIVACY
TERMS
ACCESSIBILITY

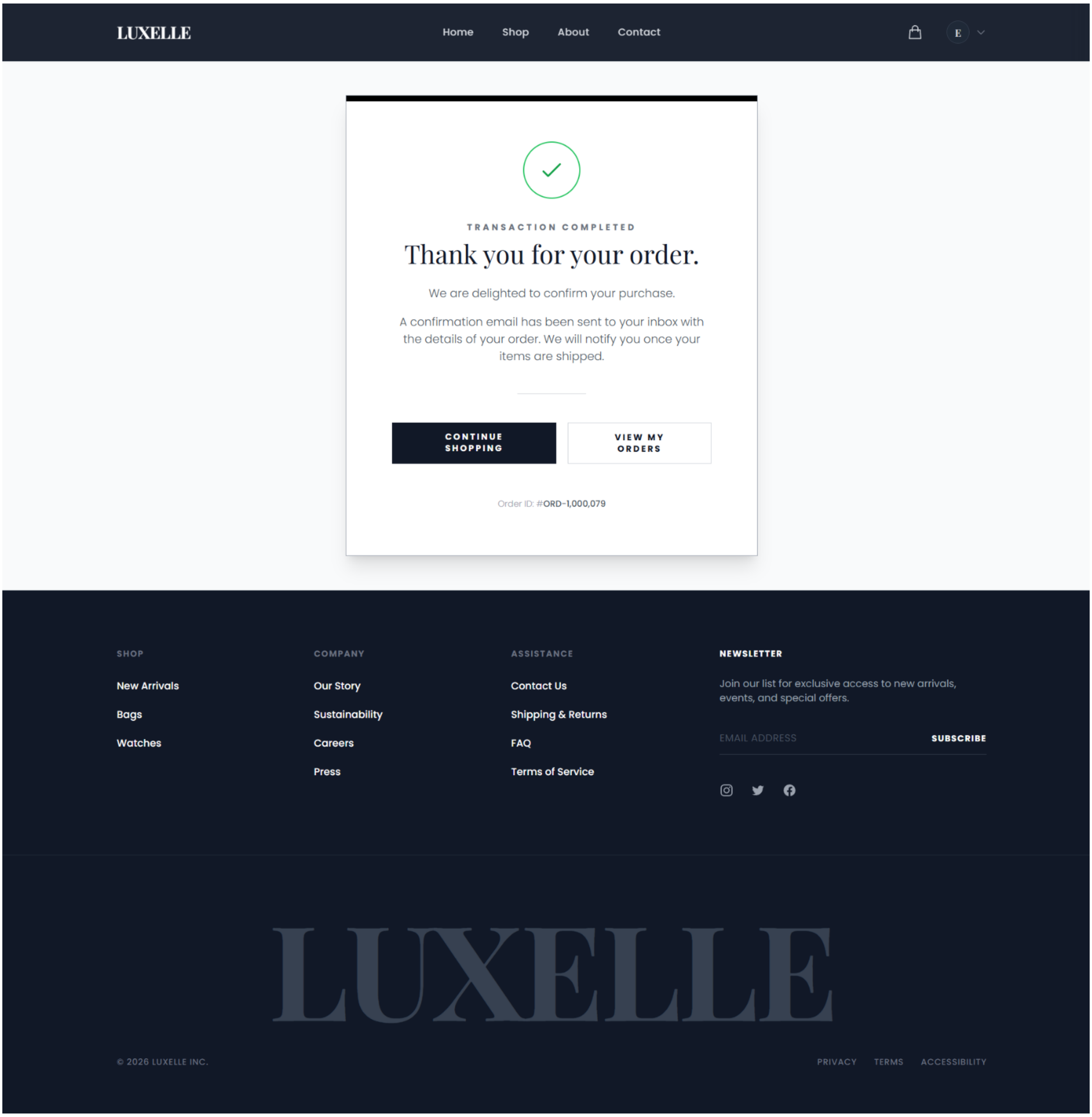
# Checkout Page:



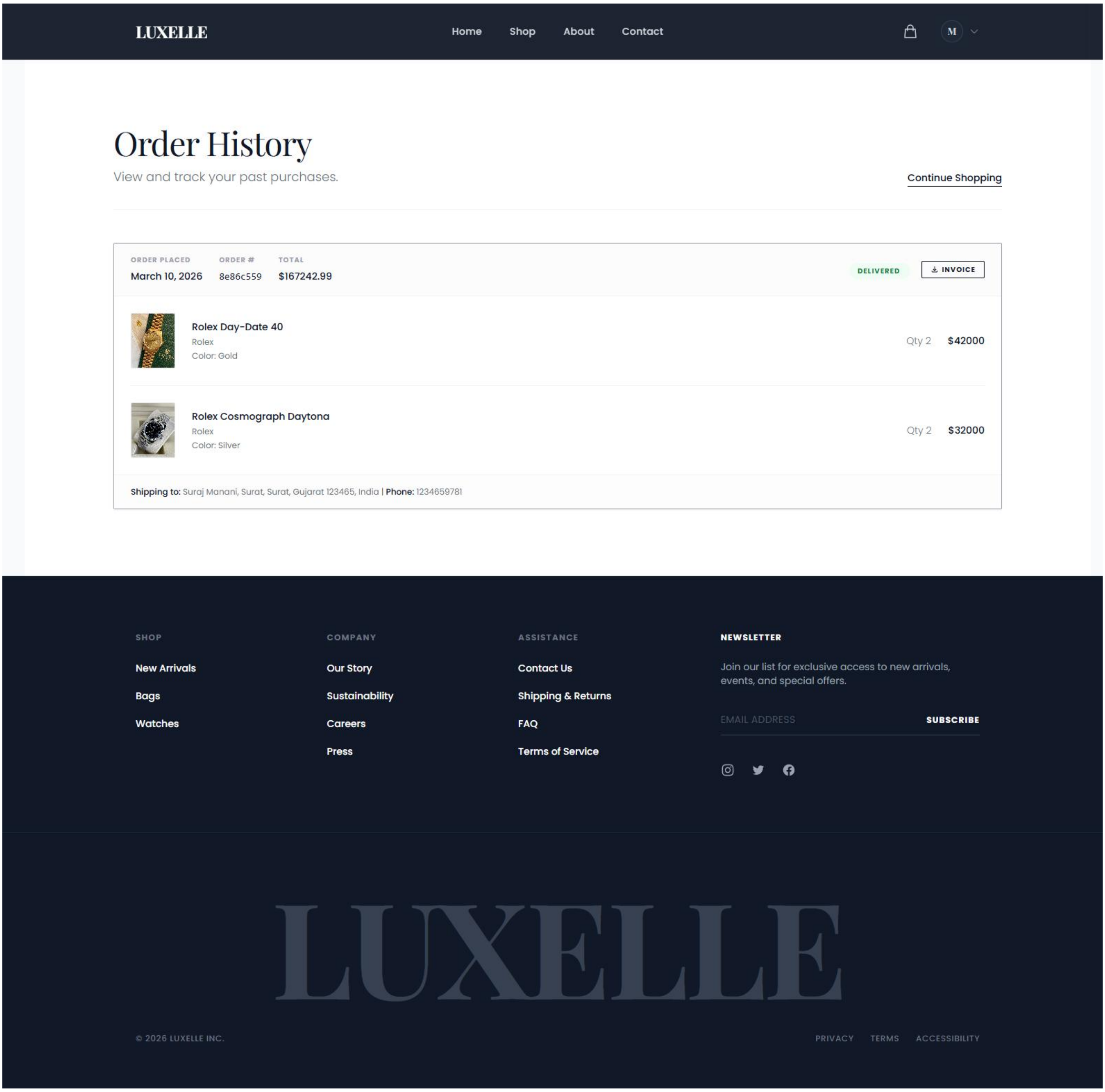
# Payment Page:



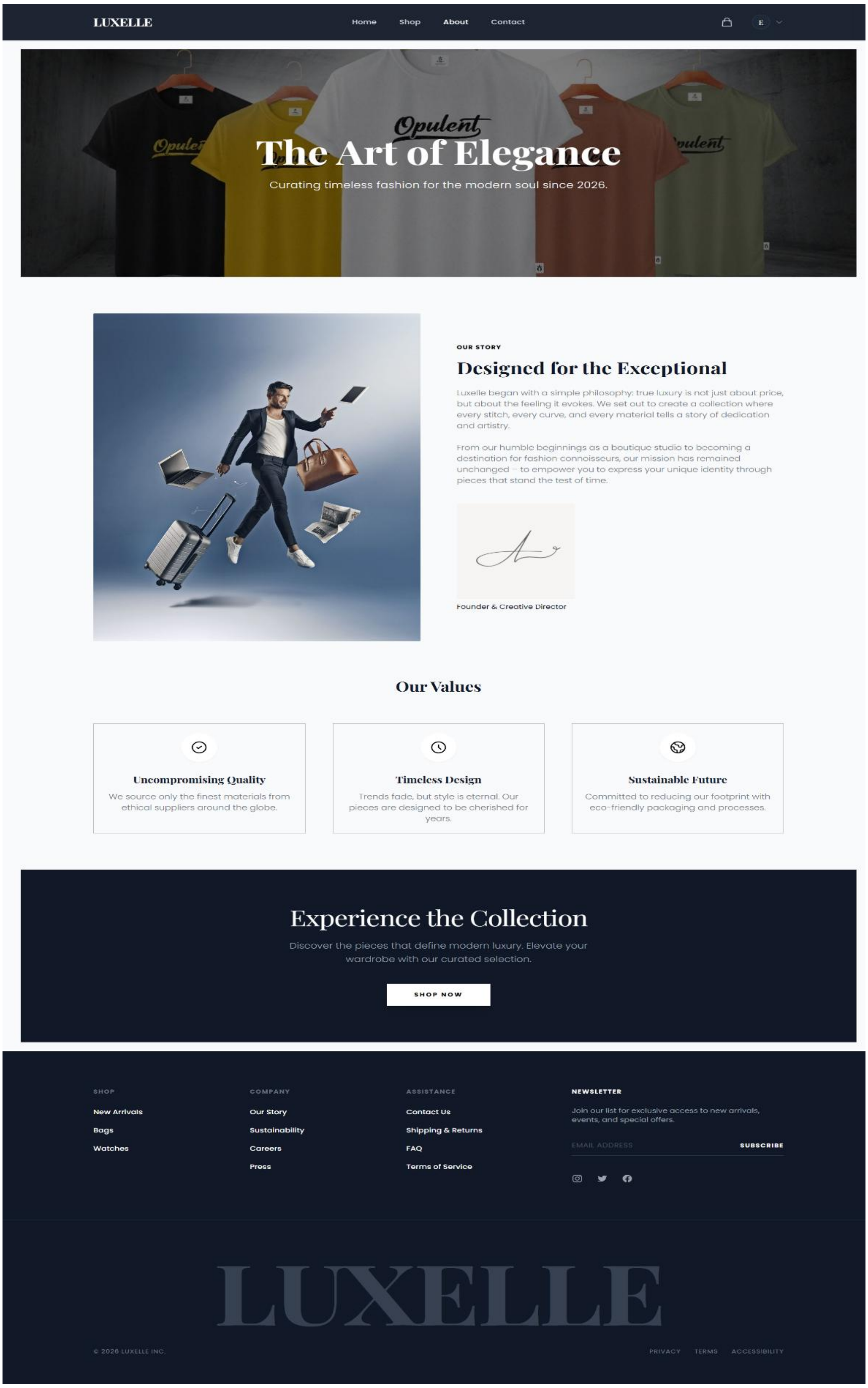
# Order Success Page:



## Order History:

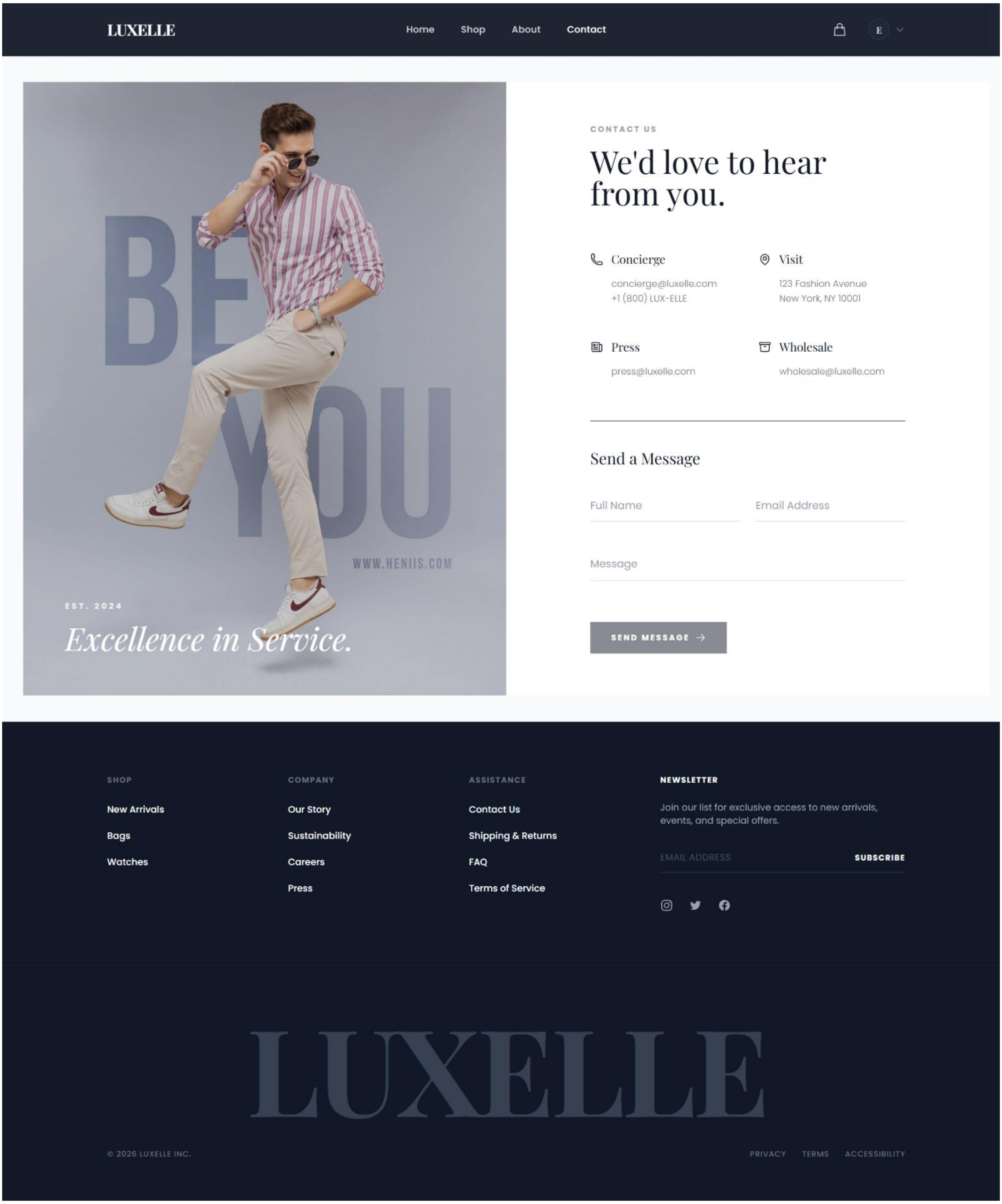


## About Page:

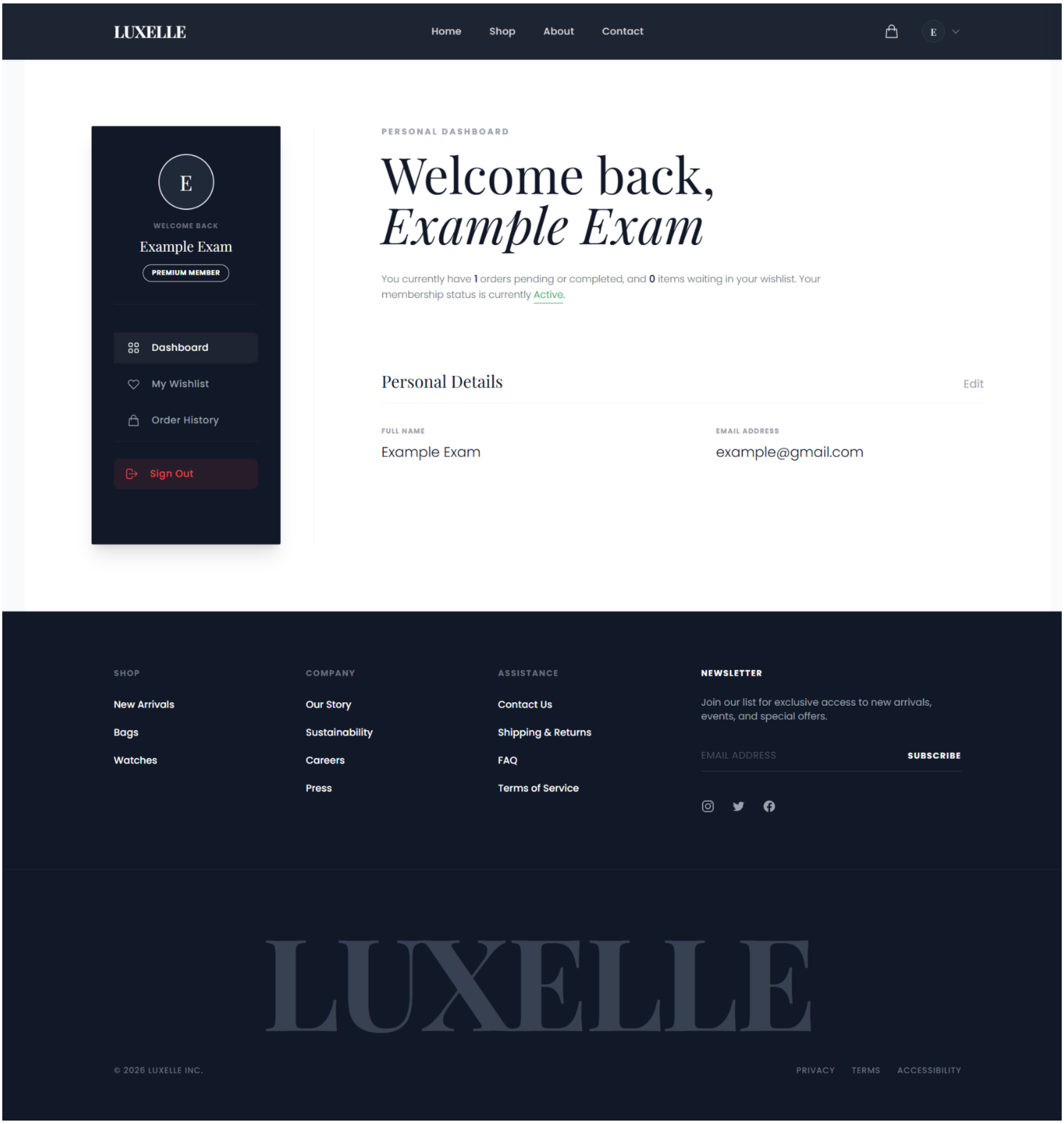


LUXELLE
Home
Shop
About
Contact
The Art of Elegance
Curating timeless fashion for the modern soul since 2026.
OUR STORY
Designed for the Exceptional
Luxelle began with a simple philosophy: true luxury is not just about price, but about the feeling it evokes. We set out to create a collection where every stitch, every curve, and every material tells a story of dedication and artistry.
From our humble beginnings as a boutique studio to becoming a destination for fashion connoisseurs, our mission has remained unchanged — to empower you to express your unique identity through pieces that stand the test of time.
Founder & Creative Director
Our Values
Uncompromising Quality
We source only the finest materials from ethical suppliers around the globe.
Timeless Design
Trends fade, but style is eternal. Our pieces are designed to be cherished for years.
Sustainable Future
Committed to reducing our footprint with eco-friendly packaging and processes.
Experience the Collection
Discover the pieces that define modern luxury. Elevate your wardrobe with our curated selection.
SHOP NOW
SHOP
New Arrivals
Bags
Watches
COMPANY
Our Story
Sustainability
Careers
Press
ASSISTANCE
Contact Us
Shipping & Returns
FAQ
Terms of Service
NEWSLETTER
Join our list for exclusive access to new arrivals, events, and special offers.
EMAIL ADDRESS
SUBSCRIBE
LUXELLE
© 2026 LUXELLE INC.
PRIVACY
TERMS
ACCESSIBILITY

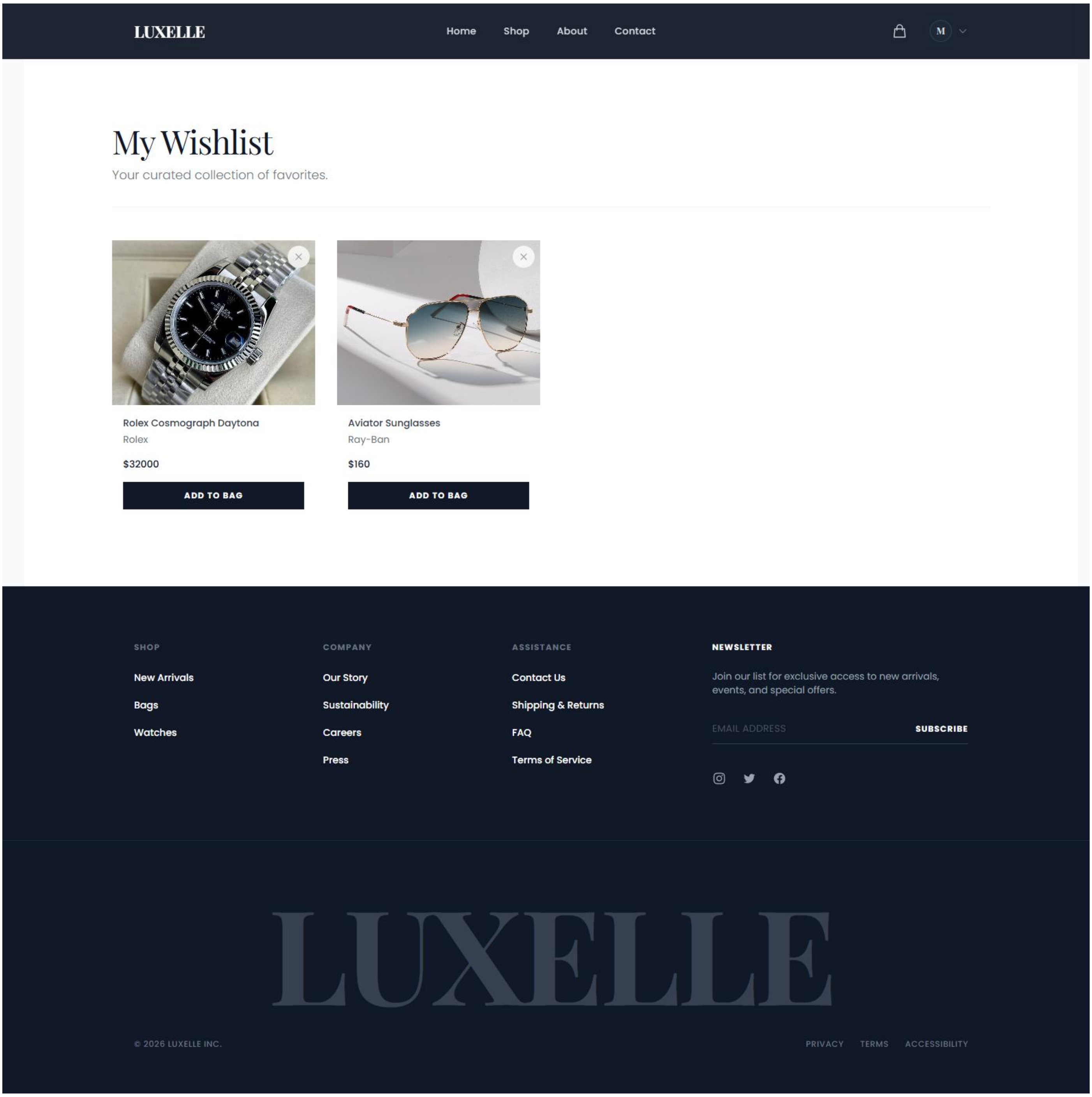
## Contact Page:



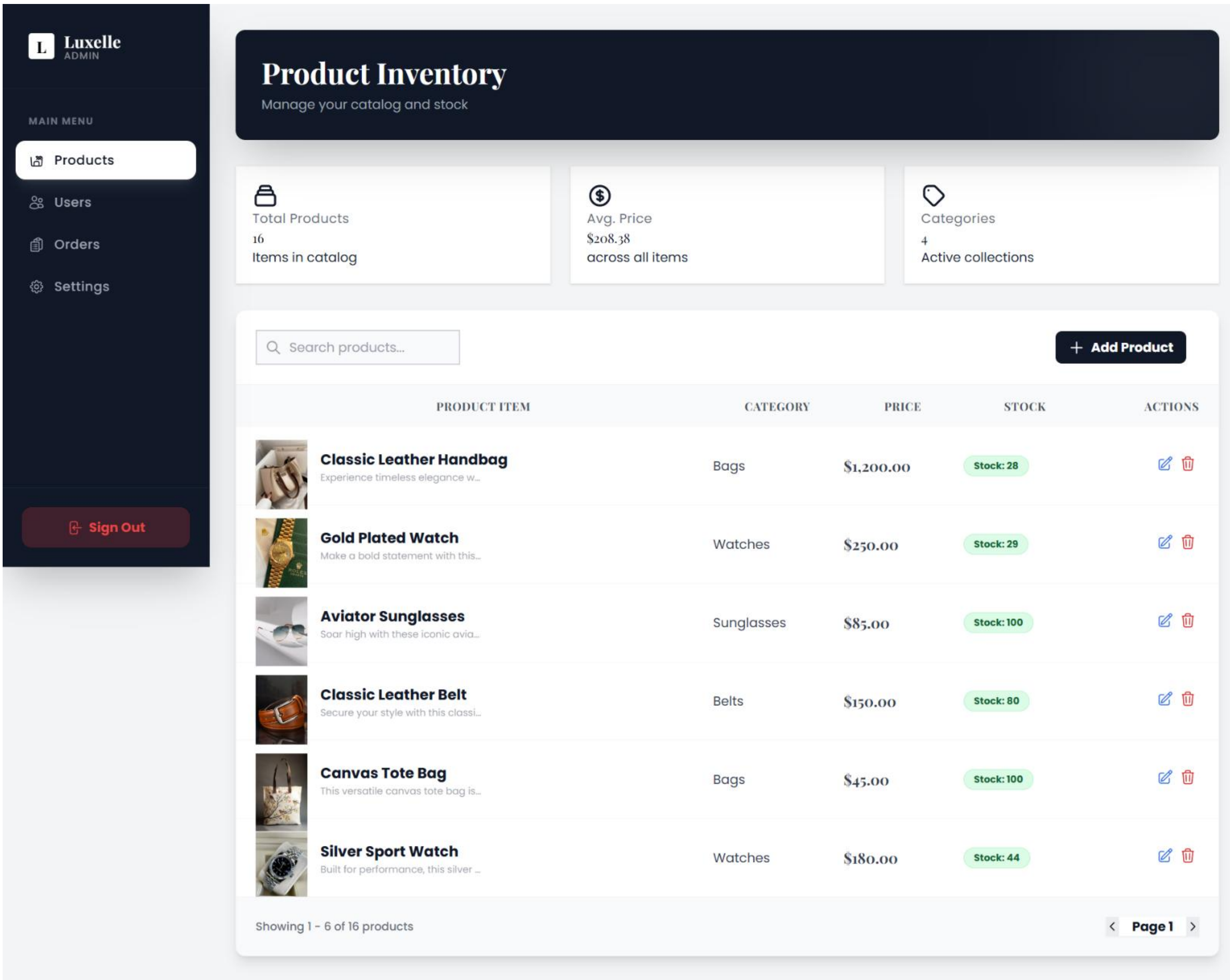
# Profile Page:



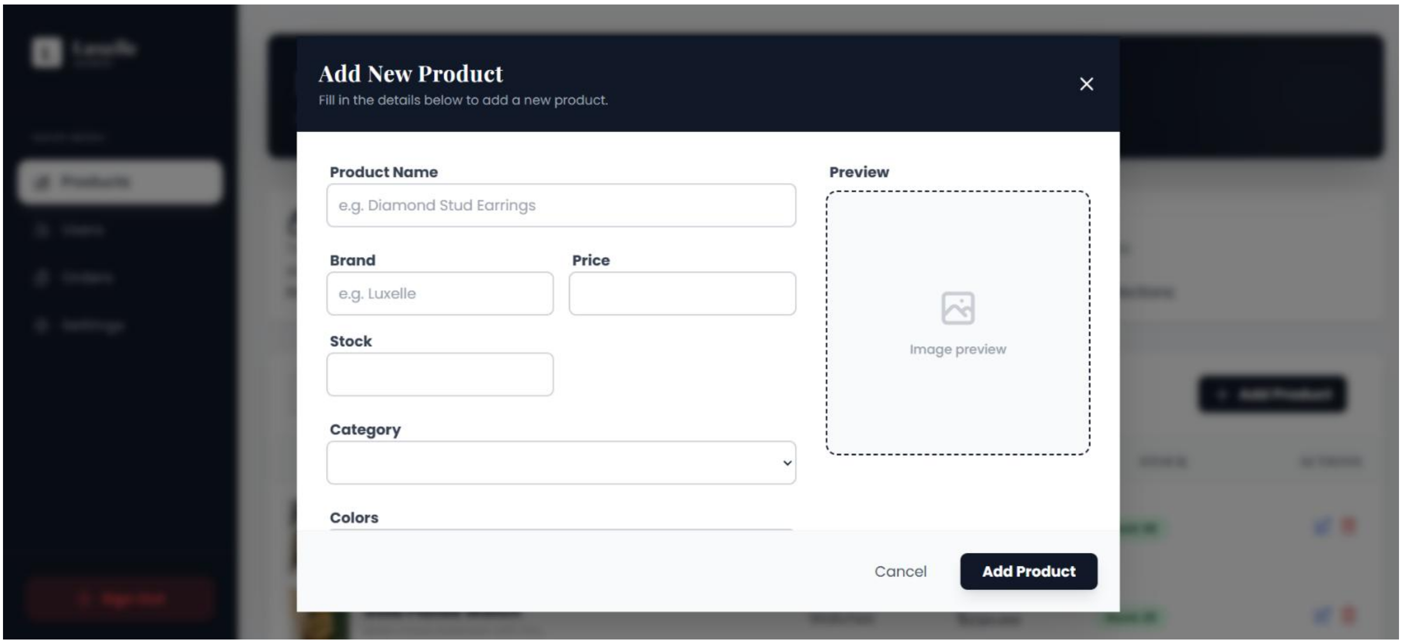
## Wishlist Page:



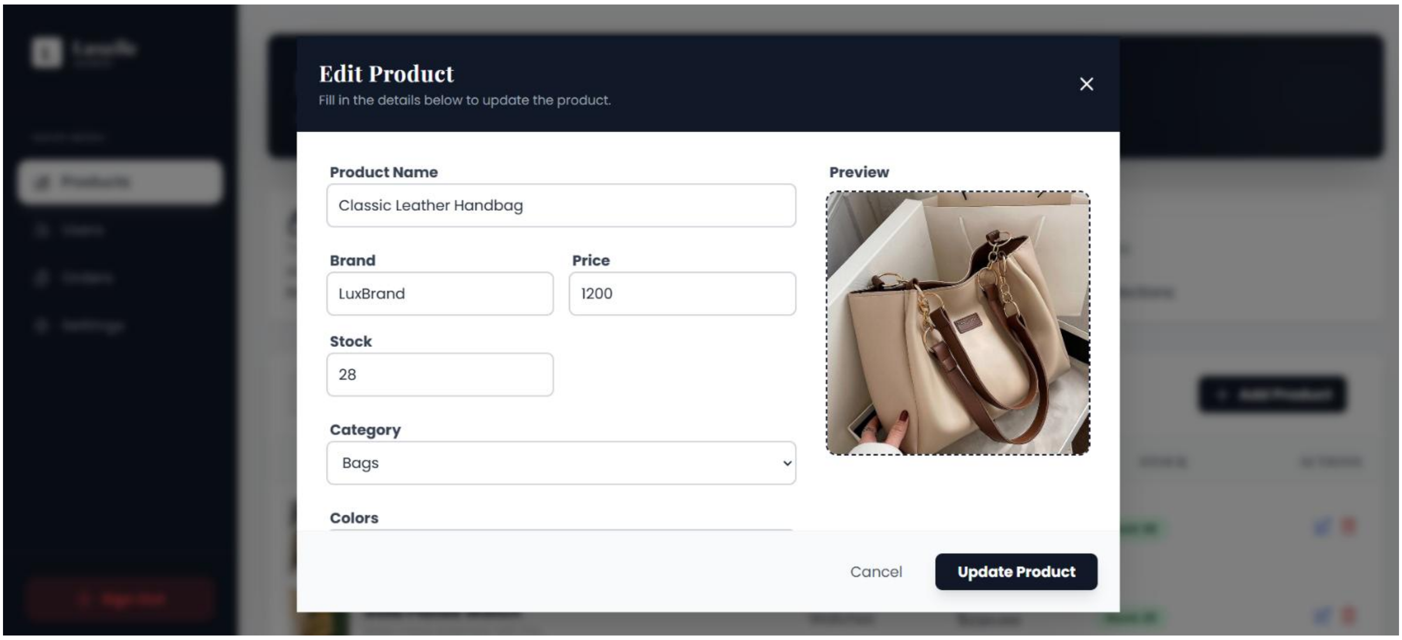
# Admin Dashboard:



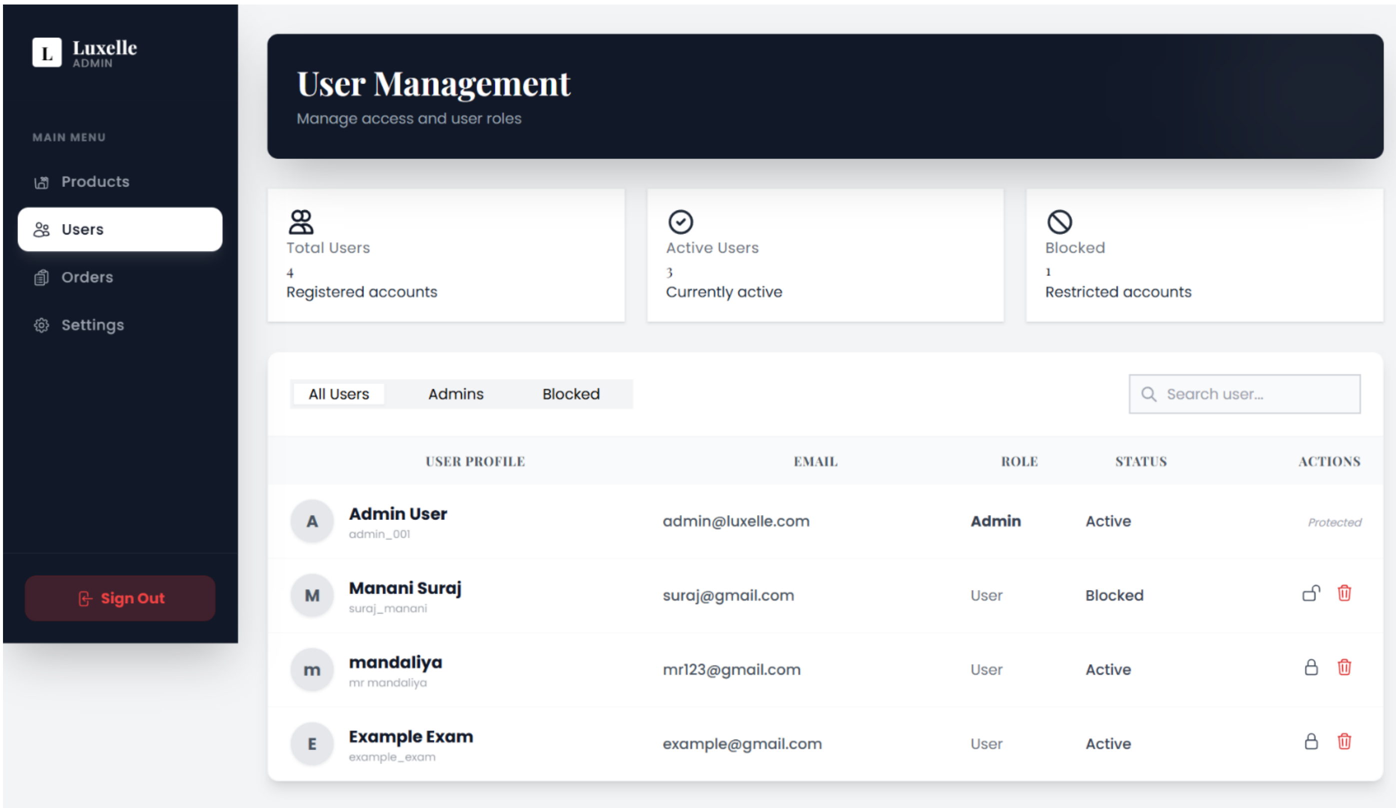
# Add Products:



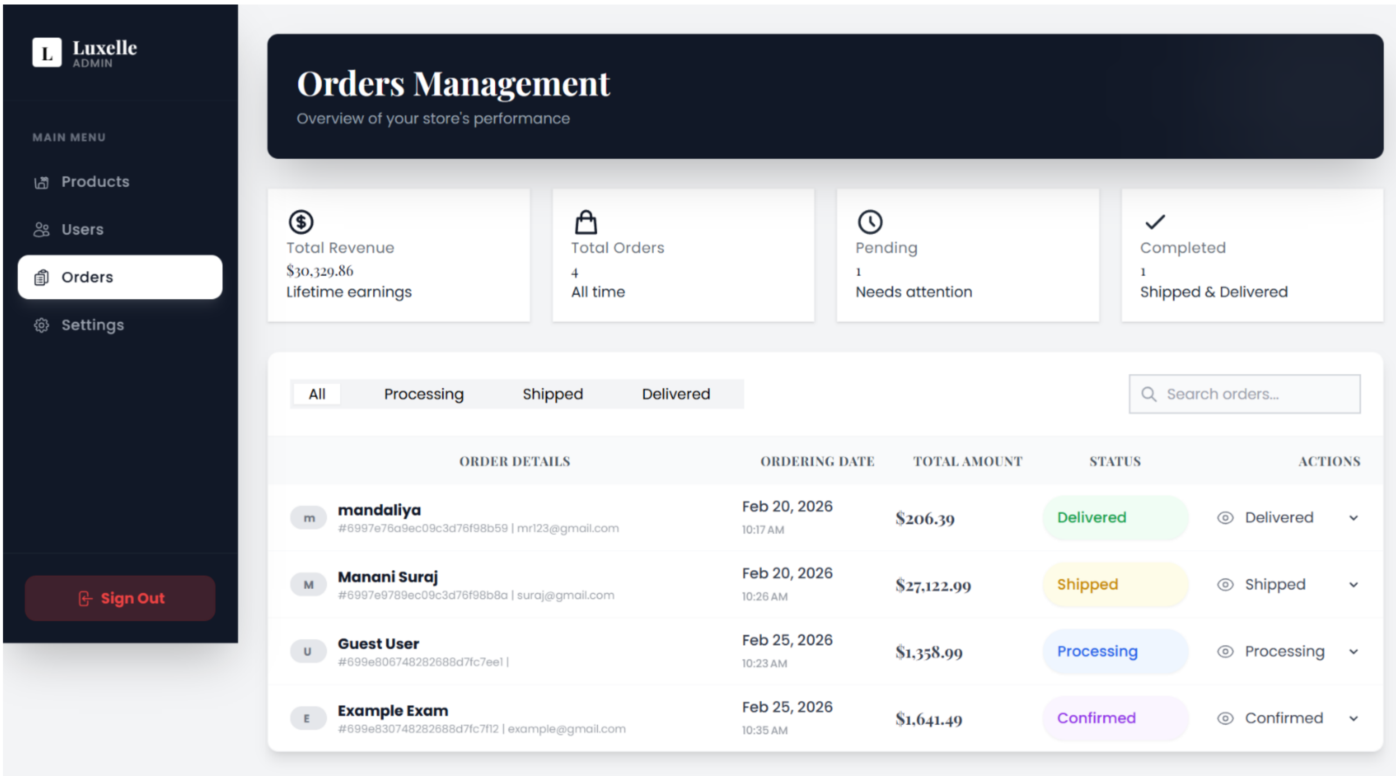

# Edit Products:



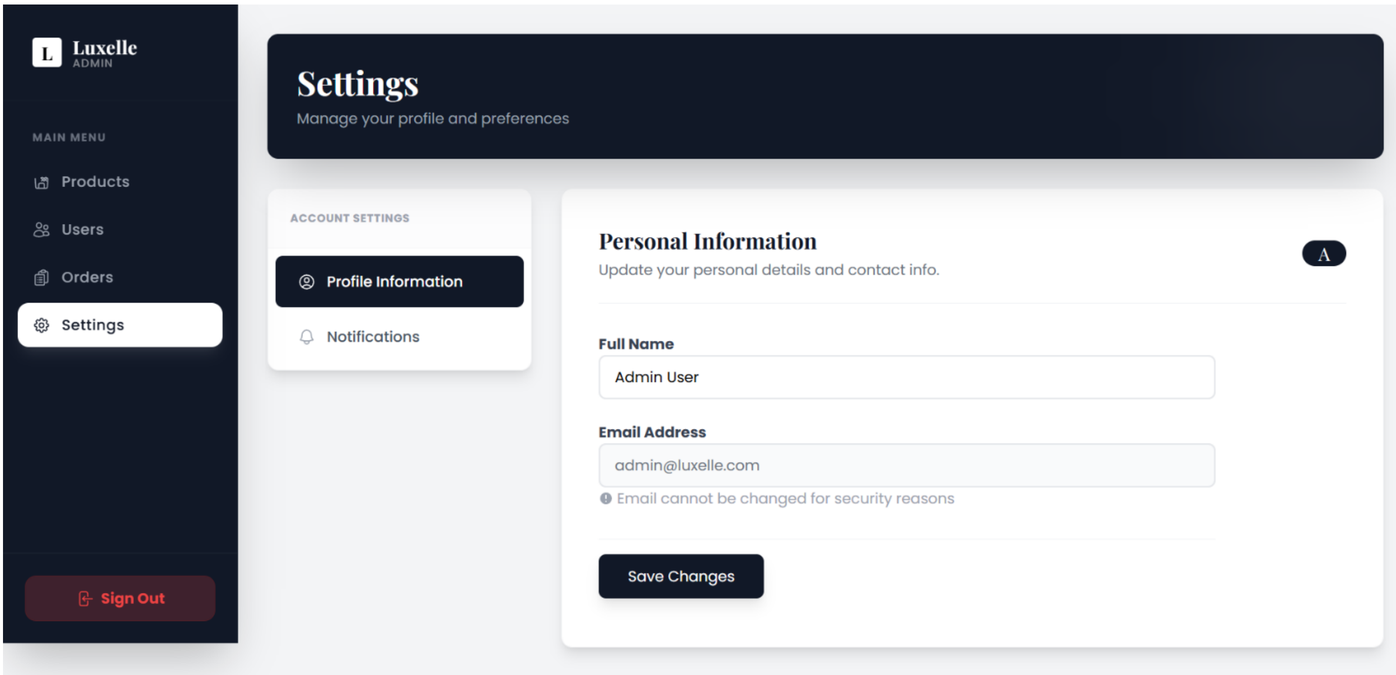
## Manage Users:




## Manage Orders:



# Setting Page:



# Chapter 4:
# System Testing

## Validations

To ensure the application is secure and reliable, "Luxelle" uses a multi-layered validation strategy. This serves as the primary method of system testing and is broken down into three layers.

### 4.1 Frontend (Client-Side) Validation

One of the most common validations is for **Required Fields**. For critical inputs like "Full Name," "Shipping Address," or "Product Name," the system enforces that these fields cannot be left empty. If a user attempts to bypass these, the submit button remains disabled, and a helpful error message appears below the specific field. This prevents the frustration of submitting a form only to have it rejected by the server, creating a smoother and more responsive interaction flow for the user.

**Email Validation** is handled with a custom validator that goes beyond checking for a simple string. The system ensures that the input follows the standard email format (user@domain.com) and specifically checks for the presence of the '@' symbol and a valid domain extension like '.com'. This reduces the likelihood of users accidentally signing up with typos in their email addresses, which is vital for account recovery and order notifications. By catching these errors early, we ensure a cleaner database of user contacts.

**Password Strength** is another critical area verified on the client side. To protect user accounts, the registration form enforces a minimum password length (e.g., 6 characters). While currently focused on length, this validation logic is extensible to include requirements for special characters or numbers in the future. By enforcing these rules at the browser level, we educate users on security best practices and ensure that the passwords created meet our baseline security standards without needing a server round-trip.

Custom logic is also applied for **Cross-Field Validation**, such as ensuring that the "Password" and "Confirm Password" fields match during registration.

This is a common pain point for users; if the fields do not match, the form is marked invalid, and a specific error message alerts the user to the discrepancy. This preventive measure saves the user from creating an account with a password they didn't intend to set, significantly reducing login issues immediately after registration.

## 4.2 Backend (Server-Side) Validation

**Schema Validation** is the primary mechanism used. Each data model (User, Product, Order) has a strict schema definition that dictates the expected data types. For example, the `price` field in a Product must always be a Number. If a request attempts to send a String or an Object for the price, Mongoose intercepts this before the database operation serves and throws a validation error. This strong typing within a NoSQL environment prevents the erratic application behavior that can arise from inconsistent data formats.

**Unique Constraints** are strictly enforced for fields that define identity. The User schema, for instance, marks the `email` and `username` fields as unique. When a registration request receives, the database checks if these values already exist. If a duplicate is found, the operation is rejected with a specific error code. This guarantees that every user account is distinct and prevents the logic errors that would occur if multiple accounts shared the same credentials.

**Required Fields** validation on the server ensures that essential data is never missing. Even if the frontend fails to catch an empty field due to a bug, the Mongoose schema requires fields like `password`, `email`, and `product name` to be present. If an API request is made without these mandatory payloads, the server responds with a `400 Bad Request` status, detailing exactly which field is missing. This protects the application logic from crashing due to `undefined` or `null` values in critical places.

Custom **Middleware Validation** is also employed for more complex logic that spans distinct models or business rules. For example, before placing an order, middleware can verify that the products in the cart are actually in stock. This goes beyond simple format checking and validates the *state* of the data. If the stock is insufficient, the validation logic halts the checkout process and returns an informative error to the user, preventing the sale of out-of-stock items.

Finally, the backend handles **Error Formatting** to ensure security and usability. When a validation error occurs, the server does not simply crash or return a raw stack trace, which could leak sensitive system information. Instead, it catches validation exceptions and transforms them into a structured JSON response. This standardized error format allows the frontend to parse the issue and display a user-friendly message, completing the feedback loop between the server's security requirements and the user's interface.

## 4.3 Authentication & Authorization Validation

The validation of these tokens occurs via **Authentication Middleware**. For every request affecting a protected route (like viewing a profile or placing an order), this middleware intercepts the HTTP request. It extracts the token from the `Authorization` header and creates a cryptographic verification using a secret key stored on the server. If the token is valid and unexpired, the request is allowed to proceed; otherwise, it is immediately rejected with a `401 Unauthorized` status.

**Authorization** goes a step further by validating user roles. Not all authenticated users have the same privileges. The middleware checks the `isAdmin` flag embedded in the verified token payload. If a standard user attempts to access an admin-only route, such as adding a product or viewing all users, the system identifies the permission mismatch and blocks the request with a `403 Forbidden` response. This Role-Based Access Control (RBAC) is essential for protecting sensitive business operations.

**Guard Validation** on the frontend complements the backend security. Angular Route Guards implement the `CanActivate` interface to check for a valid token before even loading a route. If a user tries to navigate to `/admin` without being logged in as an admin, the guard validates their state client-side and redirects them to the login page. This provides a better user experience by preventing them from seeing a "Permission Denied" page, although the ultimate security enforcement always resides on the server.

Finally, the system validates the **Token Lifespan**. Tokens are issued with a specific expiration time (e.g., 1 day). The validation logic automatically rejects expired tokens, forcing users to re-authenticate periodically. This limits the window of opportunity for an attacker if a token were effectively stolen. By combining these layers—stateless tokens, role checks, password hashing, and expiration policies—Luxelle ensures a secure environment for all transactions.

## 4.4 Manual Testing

Manual testing serves as a critical verification step in the Luxelle development lifecycle. It involves human testers interacting with the application just as a real user would, to identify issues that automated scripts might miss. This process validates not only the functional correctness of features—like whether a meaningful button clicks—but also the overall "look and feel," usability, and visual consistency of the interface. The goal is to catch logic errors, UI glitches, and workflow bottlenecks before the product reaches the final user.

| Test Case ID | Test Description | Expected Result | Status |
|---|---|---|---|
| **TC-01** | User Registration | User account is successfully created in the database and the user is redirected to the login page. | **Pass** |
| **TC-02** | User Login (Valid Credentials) | Authentication token is generated and the user is redirected to the dashboard. | **Pass** |
| **TC-03** | Add Product to Cart | Selected product is added to the cart, cart counter updates, and item persists. | **Pass** |
| **TC-04** | Place Order | Order record is created in the database and the cart is cleared. | **Pass** |
| **TC-05** | User Login (Invalid Credentials) | System displays an error message: "Invalid Credentials". | **Pass** |
| **TC-06** | Admin Login | Admin user is successfully redirected to the admin dashboard. | **Pass** |
| **TC-07** | Create Product (Admin) | Newly created product appears in both the shop interface and the database. | **Pass** |
| **TC-08** | Update Order Status (Admin) | Order status is successfully updated (e.g., Pending → Shipped). | **Pass** |
| **TC-09** | Add Address | New address details are successfully saved to the user's address book. | **Pass** |
| **TC-10** | Search Product | Relevant products matching the search query are displayed. | **Pass** |

# Chapter 5:
# Result

## 5.1 Result

The implementation of Luxelle has yielded a high-performance, aesthetically superior e-commerce platform that successfully fulfills its primary technical and business requirements. The MERN stack has proven to be an exceptional choice, providing the necessary speed and responsiveness required for a modern shopping experience. The resulting application features a seamless Single Page Application (SPA) flow, where users can navigate from product discovery to final checkout without the friction of page reloads. This technical achievement has directly contributed to a more engaging and professional user interface that reflects the "Luxury First" design philosophy.

From a functional standpoint, the platform has successfully integrated all core modules, including secure user authentication, a dynamic and filterable product catalog, and a robust order processing engine. The administrative dashboard provides a centralized command center for business operations, allowing for real-time inventory and order management. The integration of JSON Web Tokens (JWT) and Bcrypt has ensured a secure environment for user data, meeting all initial security objectives. The successful execution of manual test cases across both client and server layers confirms that the system is stable and ready for a production-like environment.

The visual results are equally impressive, leveraging Tailwind CSS to create a bespoke design that is both modern and exclusive. The use of generous whitespace, sophisticated typography, and high-resolution imagery has created a digital storefront that stands apart from generic e-commerce templates. This aesthetic excellence is maintained across various devices, with a responsive layout that ensures a consistent brand experience for mobile, tablet, and desktop users. The successful balance between visual richness and high-speed performance is a key success metric for the project, proving that premium design doesn't have to come at the cost of technical optimization.

Strategically, Luxelle has provided a scalable foundation for future brand growth. The decoupled architecture and modular codebase mean that new features can be added with minimal disruption to existing functionality. The platform's ability to handle complex product schemas and high volumes of user data ensures that it can grow alongside the business's inventory and customer base. This forward-looking design approach has resulted in a "living" system that is not just a snapshot of current needs but a flexible infrastructure capable of adapting to future technological trends and market demands.

The administrative efficiency results show a significant improvement over manual or legacy systems. By centralizing product, user, and order management into a single, intuitive dashboard, the time required for daily operational tasks has been drastically reduced. Automatic inventory updates and clear order status tracking have minimized the risk of human error, leading to a more reliable fulfillment process. This operational streamlined has empowered administrators to spend less time on technical maintenance and more time on high-value business activities such as marketing, curation, and customer relationship management.

In conclusion, the Luxelle project has achieved a harmonious integration of form and function. It stands as a proof-of-concept for how modern web technologies can be orchestrated to build a professional-grade digital business. The positive results across performance, design, security, and administration validate the technical choices made during the planning phase. Luxelle is not just a functional e-commerce site; it is a premium digital asset that provides a solid, scalable, and secure platform for success in the competitive world of high-fashion retail.

# Chapter 6:
# Future Enhancements

## 6.1 Future Enhancements

Future iterations of **Luxelle** will focus heavily on deepening the relationship between the brand and the customer through Enhanced User Interactions. A primary feature planned for this phase is a comprehensive **User Review System**. By allowing verified purchasers to rate products and leave detailed text reviews, we populate the site with social proof. This not only aids other customers in making informed decisions but also builds trust and community around the brand, which is essential for a fashion platform where community opinion is highly valued.

We also plan to implement a **Live Chat** and Customer Support module. In the world of luxury retail, high-touch service is expected as part of the premium experience. Integrating a real-time chat widget, possibly powered by a hybrid of AI bots for instant answers and human agents for complex queries, will drastically reduce support resolution times. This immediate assistance can be the deciding factor for a customer hesitating at checkout, thereby directly improving conversion rates and fostering a sense of personal attention that is synonymous with high-end boutiques.

**Social Login Integration** is another key enhancement planned to reduce user friction. Currently, users must create a specific account for Luxelle. By integrating OAuth providers like Google, Facebook, and Instagram, we can allow users to sign in with a single click. This streamlines the onboarding process significantly, as users are far more likely to register if they don't have to remember yet another password. It also opens doors for deeper social integration, such as "sharing your wishlist" to social platforms, driving organic traffic to the store.

To further engage users, we will introduce a **Loyalty and Rewards Program**. This system will track user spending and engagement patterns, awarding points that can be redeemed for exclusive discounts or early access to new collections. Gamifying the shopping experience encourages repeat visits and increases customer lifetime value. It turns a simple transactional relationship into a long-term emotional engagement, where customers feel recognized and valued for their loyalty to the brand over time.

Lastly, we aim to enhance **AI-Powered Recommendation Engines**. By analyzing browsing history, past purchases, and even high-level fashion trends, the system could suggest items that truly match the individual's style. Implementations like "Shop the Look"—where an entire outfit is suggested based on a single item in the cart—can significantly increase the average order value. This personalization makes the platform feel less like a catalog and more like a personal stylist, providing a sophisticated service that is expected in the premium sector.

Beyond these user-facing features, we plan to expand Luxelle's technical reach through **Mobile Native Applications**. Using frameworks like Ionic or React Native, we can leverage the existing MERN backend to provide a specialized app experience for iOS and Android. This allows for features like push notifications for sales and updates, as well as a more "on-the-go" shopping experience. This expansion ensures that Luxelle remains a truly omni-channel platform, meeting the customer wherever they are with a consistent and high-quality brand experience.

# Chapter 7:
# Bibliography

# 7.1 Conclusion

Luxelle successfully demonstrates the power and flexibility of a modern, scalable e-commerce architecture. By leveraging the MEAN stack (MongoDB, Express.js, Angular, Node.js), the project delivers a responsive and highly interactive user experience that meets the high standards of the fashion retail industry. The separation of concerns between the client-side SPA and the server-side REST API ensures code maintainability and allows for independent scaling of frontend and backend resources, a critical factor for growing businesses. This technical journey has proven that custom, high-end commerce is achievable through the strategic application of open-source technologies.

The project highlights the effectiveness of using TypeScript and Strongly Typed Schemas. The integration of Mongoose for database modeling and Angular's typed environment provides a layer of safety and predictability that is often missing in pure JavaScript applications. This results in fewer runtime errors and a more robust development process. The system successfully implements core e-commerce functionalities—from product discovery and secure authentication to complex order processing—proving the technical viability of the chosen stack for real-world enterprise needs.

Furthermore, the design philosophy of "Luxury First" proves that technical functionality need not come at the expense of aesthetics. The seamless integration of Tailwind CSS allows for a bespoke design that reinforces brand identity. The user interface is not just a wrapper for data; it is an active participant in the sales process, guiding users intuitively through the funnel. This balance of form and function is what sets Luxelle apart from generic solutions, creating a digital environment that is as prestigious as the physical products it represents.

From a learning perspective, this project serves as a comprehensive case study in Full-Stack Development. It touches upon every layer of the application lifecycle: database design, API architecture, frontend state management, security protocols, and user interface design. It demonstrates how disparate technologies can be orchestrated to build a cohesive, professional-grade solution that addresses real-world business needs. The experience has yielded deep insights into the challenges of performance optimization, data security, and creating a scalable architecture in the modern web era.

Looking forward, the modular architecture of Luxelle lays a solid foundation for future growth. The identified limitations and planned enhancements map out a clear path for evolution. Whether it involves integrating AI for personalization, adding real-time communication tools, or deploying to a serverless cloud infrastructure, the current codebase is adaptable and ready to support these advanced features. The project concludes with a working, high-quality asset that is ready to be expanded into a market-ready platform for any premium brand.

In conclusion, Luxelle is more than just a coursework project; it is a proof-of-concept for a modern digital business. It successfully meets its primary objectives of elevating brand identity, streamlining operations, and providing a secure, engaging shopping environment. It stands as a testament to the capabilities of modern web development frameworks and offers a robust starting point for any future e-commerce endeavors. The successful marriage of style and performance achieved here sets a high bar for what a focused, well-architected fashion platform can and should be.

# 7.2 References


- **Angular Documentation**
  - https://angular.dev/
- **Node.js Documentation**
  - https://nodejs.org/
- **Tailwind CSS Documentation**
  - https://tailwindcss.com/
- **Mongoose Documentation**
  - https://mongoosejs.com/
- **MongoDB Atlas Guide**
  - https://www.mongodb.com/cloud/atlas